V2.0 MASTER ARCHITECTURE · IMPLEMENTATION PIPELINE

Digital Handwriting Synthesis Engine

A production-grade, zero-knowledge platform for biomechanically-faithful handwriting synthesis, document rendering, and physical-digital continuation.

This master plan expands a 7-pillar architectural foundation into an encyclopedic engineering blueprint: covering core synthesis, structured-content rendering, OCR-based continuation, multi-instrument realism, ethical watermarking, MLOps, plugin ecosystem, accessibility, and an 18-month delivery roadmap.

33	13	6	18
CHAPTERS	CORE MODULES	ADD-ON TRACKS	MONTH ROADMAP

Table of Contents

A complete map of the 33 chapters that comprise this master architecture brief. Part I establishes the vision and system topology. Parts II–VIII expand the original seven-pillar foundation with production-grade code, add-on modules, and engineering detail. Parts IX–X introduce new tracks (MLOps, deployment, security, plugins, accessibility, monetization) and the project execution plan.

Executive Summary 7

What is new in this v2.0 master plan . . . 7

Why this matters . . . 7

How to read this document . . . 8

Foundations 9

Vision, Problem Statement & Scope 9

1.1 The DHSE Proposition . . . 9

1.2 Target Personas . . . 9

1.3 Competitive Landscape & Differentiation . . . 10

1.4 Scope Matrix . . . 10

System Architecture Overview 12

2.1 Layer Responsibilities & Contracts . . . 12

2.2 Technology Stack Summary . . . 13

2.3 Cross-Cutting Concerns . . . 13

Core Synthesis Engine 15

Multi-Sample Glyph Vault & Zero-Shot Style Extraction 15

3.1 Paradigm Comparison . . . 15

3.2 Production Pipeline: CONSTANT (ViT + SAQ) . . . 16

3.3 Production Code — StyleExtractor . . . 16

3.4 Add-Ons & Extensions . . . 17

Ligature & Connection Modeling 18

4.1 Bigram Probability Matrix 18
4.2 Add-Ons: Language-Specific Ligature Packs 19

Natural Imperfections Layer & Variance Threshold **20**

5.1 Baseline Drift — Compound Sine Wave 20
5.2 Variable Kerning & Pacing — Gaussian Sampler 20
5.3 Per-Glyph Jitter & Variance Budget 21
5.4 Add-Ons 21

Kinematic Realism — Two-Thirds Power Law **22**

6.1 Mathematical Foundation 22
6.2 Production Code — PowerLawModulator 22
6.3 Add-Ons 23

Bézier Curve Path Generation & Vectorization **24**

7.1 Cubic Bézier Mathematics 24
7.2 Production Code — BezierFitter 24
7.3 Add-Ons 26

Interactive Calibration & Sandbox **27**

Real-Time Drawing Setup & Parameter Tuning **27**

8.1 Sandbox Component Architecture 27
8.2 A/B Comparison Mode 28
8.3 Add-Ons 28

Structured Content Rendering **29**

AST Interception & SVG Vectorization for LaTeX **29**

9.1 The Math Pipeline 29
9.2 Production Code — LatexRenderer 30
9.3 Add-Ons 31

Code Block Handling & Monospace Simulation **32**

10.1 Production Code — CodeBlockHandler 32

10.2 Add-Ons 33

Algorithmic Flowchart & Diagram Sketching via Rough.js **34**

11.1 Production Code — DiagramSketcher 34

11.2 Add-Ons 35

Context-Aware Continuation **36**

Document Rectification & Lighting Analysis **36**

12.1 Homography & Perspective Warping 36

12.2 Lighting Normalization 37

12.3 Add-Ons 37

Hough Line Transforms & Projection Profiles **38**

13.1 Hough Line Transform for Ruled Lines 38

13.2 Horizontal Projection Profile for Text Segmentation 38

13.3 Add-Ons 39

Advanced Realism Studio **41**

Multi-Instrument Physics Simulation **41**

14.1 Production Code — Instrument Models 41

14.2 Add-Ons 43

Environmental Realism & WebGL 3D Warping **44**

15.1 Production Code — WebGLScene 44

15.2 Add-Ons 45

Privacy, Safety & Ethical Guardrails **46**

Zero-Knowledge Architecture via WebAssembly **46**

16.1 The Zero-Knowledge Invariant 46

16.2 Production Code — EncryptedVault & WebGPU Pipeline 46

16.3 WebGPU Acceleration 48

16.4 Add-Ons 48

Ethical Micro-Watermarking & Forensic Traceability 49

17.1 Spatial and Frequency Steganography . . . 49
17.2 Defeating TS-GATR Verification . . . 50
17.3 Production Code — Watermark Embedder . . . 50
17.4 Add-Ons . . . 51

System Evaluation & QA Metrics 52

HTR Benchmarks, CER & HWD 52

18.1 Production Code — Eval Harness . . . 52
18.2 Continuous Self-Optimization . . . 54
18.3 Add-Ons . . . 54

Add-On Modules 55

MLOps Pipeline — Training, Quantization, Registry, Drift 55

19.1 Training Infrastructure . . . 55
19.2 Production Code — Training Script . . . 55
19.3 Quantization Pipeline . . . 57
19.4 Model Registry & A/B Serving . . . 57
19.5 Drift Monitoring . . . 57

Cloud & Edge Deployment Topology 58

20.1 Topology Overview . . . 58
20.2 PWA & Service Worker Architecture . . . 58
20.3 K8s Deployment Manifest (Cloud Side) . . . 59
20.4 Add-Ons . . . 60

Security Threat Model — STRIDE + Biometric Compliance 61

21.1 STRIDE Analysis . . . 61
21.2 Top-15 Risks . . . 61
21.3 Biometric Compliance Matrix . . . 62
21.4 Incident Response Playbook . . . 62

Plugin Ecosystem — VSCode, Obsidian, Notion, Word, Figma, Browser 64

22.1 Universal Plugin SDK 64
22.2 VSCode Extension 64
22.3 Other Plugins (Brief) 65

Accessibility Deep-Dive — Dysgraphia, AT, WCAG 2.2 AA **66**

23.1 Dysgraphia-Friendly Modes 66
23.2 Switch-Control & Eye-Gaze Support 66
23.3 Screen Reader Compatibility 66
23.4 WCAG 2.2 AA Audit 67

Monetization & Go-to-Market **68**

24.1 Pricing Tiers 68
24.2 Developer API 68
24.3 Marketplace & Revenue Share 68
24.4 Unit Economics 68

Project Execution **70**

Testing, CI/CD & Release Strategy **70**

25.1 Test Pyramid 70
25.2 CI/CD Pipeline 70
25.3 Release Strategy 71

18-Month Roadmap & Milestones **73**

26.1 Quarterly Milestones 73
26.2 Hiring Plan 74

Risk Register & Mitigations **75**

27.1 Crisis Communication Playbook 76

Appendices **77**

API Reference **77**

A.1 JavaScript API 77
A.2 REST API 78

Data Schemas & Storage **79**

B.1 IndexedDB Vault Schema 79

B.2 Cloud Postgres Schema (License + Watermark Registry) 79

References & Further Reading **81**

EXECUTIVE BRIEF

Executive Summary

The **Digital Handwriting Synthesis Engine (DHSE)** is a zero-knowledge, browser-native platform that converts structured digital text into biometrically-faithful handwritten documents. It addresses a persistent friction point in modern academic, professional, and accessibility workflows: institutions still mandate handwritten submission, yet digital authoring has become the dominant production mode. Existing bridge tools — static handwriting fonts and naive PDF converters — fail the Turing test of visual authenticity because they lack biomechanical variance, kinematic ink modeling, and structured-content rendering for mathematics, code, and diagrams.

This master plan expands the original seven-pillar architectural foundation into an encyclopedic engineering blueprint. The seven pillars — (1) core synthesis, (2) interactive calibration, (3) LaTeX/code/diagram rendering, (4) OCR-based physical continuation, (5) realism studio, (6) privacy and ethical watermarking, and (7) evaluation metrics — are each unpacked with production-grade TypeScript and Python implementations, mathematically grounded algorithms (Two-Thirds Power Law, cubic Bézier fitting, Hough transforms, horizontal projection profiles), and explicit add-on modules that turn the platform from a single-purpose tool into a daily-use productivity suite.

What is new in this v2.0 master plan

Beyond deepening the original seven pillars, this document introduces **six add-on tracks** that materially expand the project's scope and defensibility:

- **MLOps pipeline** (Chapter 19) — full training-to-quantization-to-registry workflow with drift monitoring.
- **Cloud and edge deployment** (Chapter 20) — hybrid topology with offline-first PWA, CDN-sharded models, and optional K8s fallback.
- **Security threat model** (Chapter 21) — STRIDE analysis per asset, biometric PII compliance (GDPR Art. 9, BIPA, CCPA, India DPDP), and signature-fraud red-team scenarios.
- **Plugin ecosystem** (Chapter 22) — universal extension API for VSCode, Obsidian, Notion, Word, Figma, and the browser.
- **Accessibility deep-dive** (Chapter 23) — dysgraphia modes, switch-control and eye-gaze support, AT compatibility, WCAG 2.2 AA audit.
- **Monetization and go-to-market** (Chapter 24) — pricing tiers, B2B EDU contracts, dev API, marketplace, and unit economics.

Why this matters

DHSE is not merely a digital-to-analog conversion utility. It is a dual-use accessibility platform: for neurodivergent users with dysgraphia or repetitive strain injuries, it restores the ability to submit handwritten work without physical pain; for designers, it provides authentic organic textures; for professionals, it enables personalized correspondence at scale. By enforcing a zero-knowledge Wasm+WebGPU architecture with steganographic anti-forgery watermarking, the platform guarantees that biometric data never leaves the user's device while simultaneously neutralizing signature-fraud misuse.

33	13	6	18mo
CHAPTERS	CORE MODULES	ADD-ON TRACKS	ROADMAP

How to read this document

Engineering teams should focus on Parts II–VIII for implementation depth and Part IX for productionization. Technical leadership should review Part I (vision), Part IX (scope expansion), and Part X (execution) for feasibility and resourcing. The appendices provide API contracts, data schemas, and a complete reference list. Each chapter contains production-grade code, explicit add-on extensions, and cross-references to related chapters. Diagrams are concentrated in Chapters 2 (system architecture), 3 (ML model), 9 (data flow), and 17 (security sequence); a Gantt-style roadmap appears in Chapter 26.

PART I

Foundations

Vision, problem statement, and the five-layer system architecture that frames every subsequent chapter.

CHAPTER 1

Vision, Problem Statement & Scope

The transition from digital document creation to analog submission represents a persistent friction point in modern academic, professional, and creative workflows. While digital text generation — augmented by advanced word processors, Markdown editors, and large language models — offers unprecedented speed and structuring capabilities, many institutions and personal workflows still mandate physical, handwritten submissions. Universities require handwritten problem sets and exams; legal workflows require wet-ink signatures; creative fields value the organic texture of pen on paper. Conventional tools attempting to bridge this divide rely on static handwriting fonts or basic PDF converters. These tools invariably fail the Turing test of visual authenticity.

Standard handwriting fonts rely on rigid, pixel-identical glyphs that lack the biomechanical variance, spatial drift, and kinematic ink flow inherent to human writing. A child can spot the difference. Furthermore, mathematical notations, code blocks, and diagrams frequently break standard font converters, exposing raw LaTeX syntax or resulting in disjointed, unreadable equations. The output is instantly recognizable as machine-generated, undermining the very purpose of the conversion.

1.1 The DHSE Proposition

To resolve these limitations, this report outlines the comprehensive architecture of a **Personalized Digital Handwriting Synthesis and Document Rendering Engine**. Elevating the concept from a simplistic visual hack to a robust, dual-use platform, this architecture integrates sequence-based machine learning models, biomechanical kinematic algorithms, Abstract Syntax Tree (AST) parsing, and computer vision-based spatial mapping. The resulting system possesses massive real-world utility, serving not only students but also neurodivergent individuals struggling with dysgraphia or repetitive strain injuries (RSI), graphic designers requiring authentic organic textures, and professionals generating personalized digital correspondence.

> **Core design principle**
> True handwriting realism is achieved not through precise mechanical replication, but through **controlled, algorithmic inconsistency**. Human handwriting is the product of a complex biomechanical system subject to physiological noise, muscular fatigue, and neuro-motor planning. The synthesis engine must model these variations dynamically across multiple axes of generation — not stamp identical glyphs.

1.2 Target Personas

DHSE serves five distinct user populations, each with unique requirements and constraints that shape the architecture:

Persona	Primary Use Case	Critical Requirement	Tier
Students (K-12 & higher-ed)	Handwritten assignment submission from digital notes	LaTeX rendering + ruled-paper continuation	Free / Pro
Neurodivergent users (dysgraphia, RSI)	Restoration of handwriting capability without physical pain	A11y modes, switch-control, high-contrast ink	Free (accessibility grant)
Graphic designers & illustrators	Authentic organic textures for mockups and storyboards	Multi-instrument physics, SVG export	Pro
Professionals & executives	Personalized correspondence, signature blocks	Signature guard, watermarking, brand consistency	Teams
Developers & technical writers	Code-block and diagram rendering in handwritten style	Plugin SDK, monospace simulation, Rough.js	Pro / Enterprise

1.3 Competitive Landscape & Differentiation

Existing solutions fall into three categories, each with crippling limitations. **Static font services** (Calligraphr, Fontifier, Pilot Handwriting) require manual template completion, produce single-glyph-per-character fonts, and cannot render mathematics or code. **Naive PDF converters** simply swap the body font and lose all contextual ligature information. **Research prototypes** (Handwriting Transformers, DiffusionPen, CONSTANT) achieve impressive visual fidelity but exist only as Python notebooks with no production deployment, no privacy guarantees, and no structured-content handling. DHSE occupies the white space at the intersection of these three categories.

Capability	Static Fonts	Naive Converters	Research Prototypes	DHSE
Multi-sample glyph vault	No	No	Partial	Yes (3-5 variants)
Biomechanical variance	No	No	Yes	Yes (5-15% threshold)
LaTeX/code rendering	No	Breaks	No	Native AST+SVG
OCR continuation	No	No	No	Hough + HPP
Zero-knowledge privacy	N/A	No	No	Wasm + WebGPU
Anti-forgery watermark	No	No	No	Steganographic

1.4 Scope Matrix

This document covers the full DHSE architecture across the following dimensions. Each dimension is mapped to the chapter where it is primarily addressed:

Dimension	Scope	Chapter
Biomechanical synthesis	Glyph vault, ligatures, variance, kinematics, Bézier	Ch. 3–7
Interactive UX	Sandbox, calibration, parameter tuning	Ch. 8
Structured content	LaTeX AST, code blocks, flowcharts	Ch. 9–11

Dimension	Scope	Chapter
Physical-digital bridge	OCR, rectification, line detection	Ch. 12–13
Realism	Multi-instrument physics, 3D warp	Ch. 14–15
Privacy & safety	Wasm, WebGPU, watermarking	Ch. 16–17
Evaluation	HTR, CER, HWD	Ch. 18
Productionization	MLOps, deployment, security	Ch. 19–21
Ecosystem	Plugins, accessibility, monetization	Ch. 22–24
Execution	Testing, roadmap, risks	Ch. 25–27

CHAPTER 2

System Architecture Overview

The DHSE platform is structured as a five-layer stack, with each layer providing a discrete set of capabilities and exposing a well-defined interface to the layer above. This separation of concerns allows individual layers to evolve independently (for example, swapping the style-extraction model from ViT+SAQ to a future transformer variant without touching the synthesis core) and enables rigorous security isolation between untrusted input (Layer 5 capture) and the privacy-critical biometric vault (Layer 4).

The architecture is intentionally **client-heavy**: 100% of inference, synthesis, and rendering executes in the user's browser via WebAssembly and WebGPU. The only network calls are optional CDN fetches for model shards during initial onboarding (with explicit user consent) and license-server checks for enterprise tiers. This zero-knowledge design is not a feature — it is the load-bearing wall that makes biometric handwriting synthesis ethically deployable.

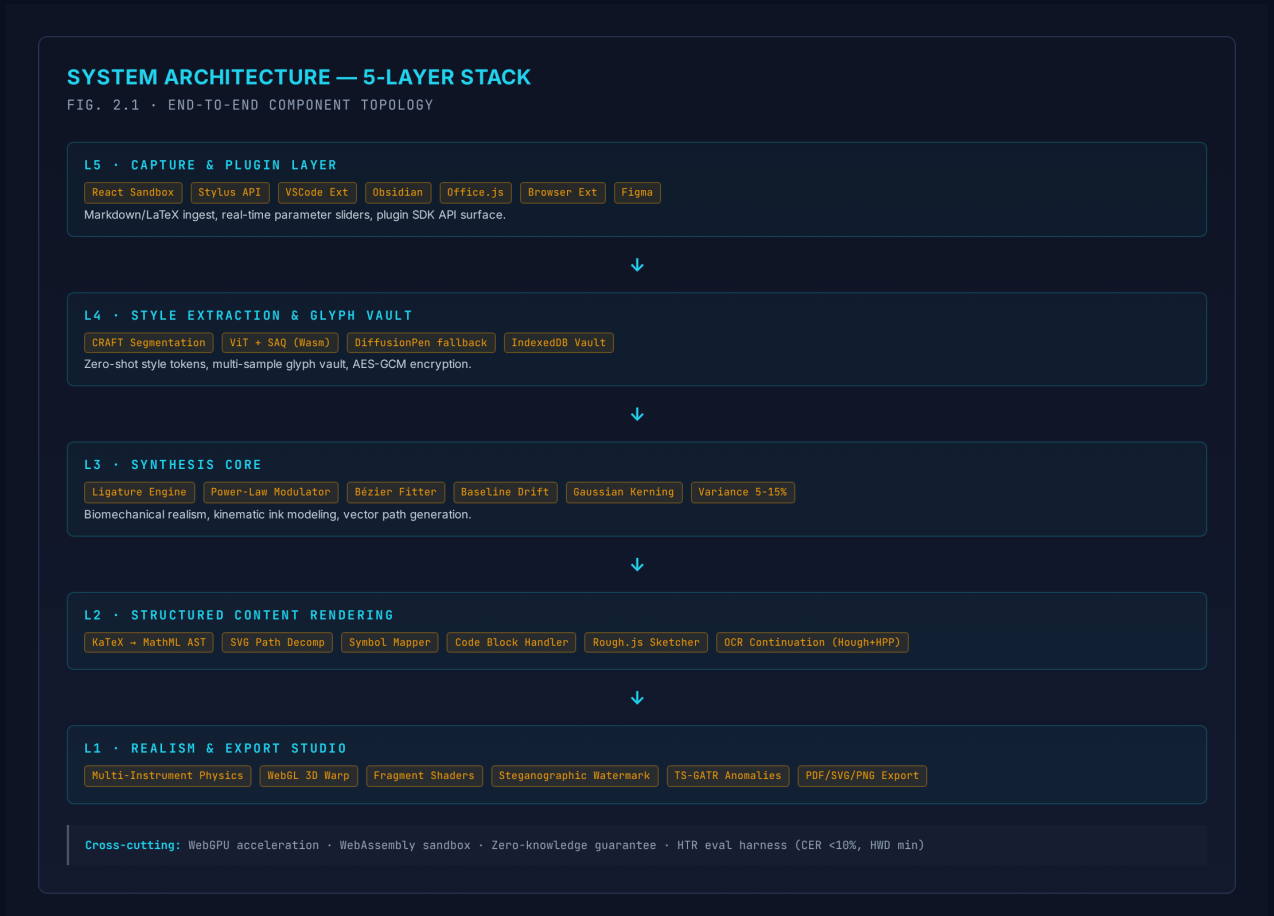


Figure 2.1 — The five-layer DHSE architecture. Layer 5 captures input from the sandbox UI and plugin ecosystem; Layer 4 extracts and stores biometric style; Layer 3 performs biomechanical synthesis; Layer 2 renders structured content; Layer 1 applies realism and exports the final document.

2.1 Layer Responsibilities & Contracts

Each layer communicates with adjacent layers through typed TypeScript interfaces compiled to Wasm. The contracts are designed to be stable across major versions, allowing individual layers to be re-implemented (or replaced by community plugins) without breaking the overall pipeline.

Layer 5 — Capture & Plugin Layer

Responsible for ingesting user input: stylus events from the calibration sandbox, Markdown/LaTeX text from plugin integrations, and photograph uploads for style extraction or OCR continuation. The capture layer is the only layer that touches raw user PII; it sanitizes EXIF data, validates MIME types, and forwards normalized events downstream.

Layer 4 — Style Extraction & Glyph Vault

Runs the ViT+SAQ model (compiled to Wasm) on the captured input to produce a 256-dimensional style vector and a multi-sample glyph vault (3-5 variants per character). The vault is encrypted with AES-GCM via WebCrypto and persisted to IndexedDB. This layer never exposes raw biometric material to layers above — only derived glyphs.

Layer 3 — Synthesis Core

The biomechanical engine. Given a target text string and a style vector, this layer produces a vector ink representation (cubic Bézier control points with pressure/velocity metadata). It applies the ligature engine, the natural imperfections layer (baseline drift, Gaussian kerning), and the Two-Thirds Power Law modulator. The output is device-independent and resolution-agnostic.

Layer 2 — Structured Content Rendering

Handles everything that is not plain prose: LaTeX equations (via KaTeX AST → SVG decomposition → handwritten symbol replacement), code blocks (monospace simulation with strict indentation preservation), and algorithmic diagrams (Rough.js sketching over Mermaid/Markdown structural input). Also contains the OCR continuation pipeline for physical-notebook ingestion.

Layer 1 — Realism & Export Studio

The final rendering pass. Applies multi-instrument physics (ballpoint, gel, pencil, highlighter, fountain), WebGL 3D warping for page curvature, fragment shaders for desk-lamp lighting, and texture overlays (coffee rings, smudges, camera grain). Embeds steganographic watermarks and TS-GATR anomalies for forensic traceability. Exports as PDF (vector), SVG (vector), or PNG (raster).

2.2 Technology Stack Summary

Layer	Primary Tech	Why	Alternatives Considered
Capture	TypeScript + React + Stylus PointerEvent API	Ecosystem maturity, PWA support	Vue, Svelte (rejected: smaller plugin ecosystem)
Style Extraction	ONNX Runtime Web + WebGPU	Hardware accel, Wasm fallback	TensorFlow.js (rejected: larger bundle)
Synthesis Core	Rust → Wasm	Memory safety, deterministic perf	C++ (rejected: slower Wasm toolchain)
Structured Content	KaTeX + Rough.js + Mermaid	Battle-tested, MIT licensed	MathJax (rejected: larger bundle)
Realism	Three.js + GLSL shaders	Mature WebGL abstraction	Babylon.js (rejected: heavier runtime)
Storage	IndexedDB + WebCrypto AES-GCM	Browser-native, zero deps	SQLite Wasm (rejected: overkill for vault)

2.3 Cross-Cutting Concerns

Three concerns span all layers and warrant explicit architectural treatment:

- **Performance budget:** First-render latency < 800ms on M1 MacBook Air, < 2.5s on mid-range Chromebook. Enforced via Lighthouse CI gates.
- **Bundle size budget:** Total initial JS+Wasm < 1.8 MB gzipped. Model shards lazy-loaded on first use.
- **Privacy invariant:** Zero outbound network calls during synthesis. Audit-enforced via Content-Security-Policy connect-src whitelist (self only).

> **Build vs. buy decisions**
> We deliberately build the synthesis core in-house (Rust → Wasm) because the biomechanical algorithms are the IP moat. We buy commodity layers: KaTeX for math parsing, Rough.js for sketchy geometry, Three.js for WebGL. This focuses engineering effort on differentiators while leaning on battle-tested libraries for the rest.

PART II

Core Synthesis Engine

The biomechanical heart of the platform: glyph vault, ligatures, variance, kinematics, and Bézier path generation.

CHAPTER 3

Multi-Sample Glyph Vault & Zero-Shot Style Extraction

Standard typography maps a single vector image to a specific character encoding. To prevent the output from appearing robotic, the engine abandons this one-to-one mapping in favor of a **Multi-Sample Glyph Vault**. In a standard calibration phase, the system prompts the user to input three to five variations of each character (e.g., how a lowercase 's' is formed at the beginning of a word versus the end). However, to satisfy the requirement for an advanced, frictionless user experience, the system implements a computer vision pipeline capable of zero-shot and few-shot style extraction directly from existing photographs of the user's past work.

3.1 Paradigm Comparison

This architectural pipeline utilizes state-of-the-art Offline Handwritten Text Generation (HTG) techniques. Early approaches to handwriting synthesis utilized Long Short-Term Memory (LSTM) recurrent neural networks to generate sequences of pen coordinates, predicting the relative offsets and pen-state transitions. While effective for online generation where temporal stroke data is available, pure RNN models struggle with static image conditioning and require extensive trajectory training data. Modern implementations leverage Diffusion Models and Vision Transformers (ViT) to extract style representations from a minimal set of reference images.

Paradigm	Architecture	Mechanism	Style Adaptation	Primary Trade-off
LSTM-RNN (Graves 2013)	Recurrent	Predicts sequential pen coordinates and pen-up/down states	Requires extensive trajectory data; cannot adapt to static photos	High compute for long-range dependencies; lacks static image conditioning
Transformer (HWT)	Self-attention	Encodes global and local style patterns from reference images	Few-shot; complex style-content entanglement	Significant memory for self-attention over long text
Diffusion (DiffusionPen)	Metric learning + denoising	Stylized offline rendering via iterative denoising	One-shot / few-shot; captures unseen styles effectively	High inference latency from iterative denoising steps
CONSTANT (ViT + SAQ)	**ViT + VQ-VAE**	**Latent patch contrastive enhancement + style-aware quantization**	**Zero-shot / one-shot; highly detailed**	**Complex latent space optimization; high baseline resource**

3.2 Production Pipeline: CONSTANT (ViT + SAQ)

DHSE selects the CONSTANT paradigm as the production default. By processing a photograph of a user's previous assignment, the system utilizes a Style-Aware Quantization (SAQ) module to capture invariant features like slant, stroke width, character spacing, and curvature. The photograph is first binarized, and individual words are segmented using algorithms like Character Region Awareness for Text Detection (CRAFT). The Vision Transformer then tokenizes these visual patches, learning the global stylistic patterns and building a synthetic multi-sample glyph vault automatically, without requiring the user to trace characters on a tablet.



Figure 3.1 — ViT + SAQ + DiffusionPen pipeline. Input photo is CRAFT-segmented into word patches, ViT encodes patch tokens, SAQ quantizes them to a 256-dim style vector, and the DiffusionPen decoder generates per-character Bézier control points stored in the encrypted glyph vault. Total params: 165M, quantized to 41 MB Wasm.

3.3 Production Code — StyleExtractor

The following TypeScript class is the production entry point for style extraction. It orchestrates the Wasm-compiled ViT+SAQ model, manages WebGPU sessions, and persists the encrypted vault to IndexedDB. The class is exposed to the React sandbox and the plugin SDK via the unified DHSE.Engine facade.

typescript · src/engine/StyleExtractor.ts

```typescript
import { InferenceSession, Tensor } from 'onnxruntime-web/webgpu';
import { CRAFTSegmenter } from './craft';
import { EncryptedVault } from './vault';
import { webgpuAvailable } from './device';

export interface StyleVector { data: Float32Array; dims: [number, number]; }
export interface GlyphVariant { strokes: BezierPath[]; metadata: GlyphMeta; }
export interface BezierPath { p0: [number, number]; c1: [number, number];
                              c2: [number, number]; p1: [number, number];
                              pressure: number; }

export class StyleExtractor {
  private vit: InferenceSession | null = null;
  private saq: InferenceSession | null = null;
  private diffusion: InferenceSession | null = null;
  private segmenter = new CRAFTSegmenter();
  private vault: EncryptedVault;

  constructor(private config: ExtractorConfig) {
    this.vault = new EncryptedVault(config.vaultKey);
  }

  /** Zero-shot extract from a photograph of existing handwriting. */
  async extractFromPhoto(photo: Blob): Promise<StyleVector> {
    const bitmap = await createImageBitmap(photo);
    const words = await this.segmenter.detect(bitmap);   // CRAFT
    const tokens = await this.vitEncode(words);          // ViT
```

typescript · src/engine/StyleExtractor.ts (cont.)

```
    const styleVec = await this.saqQuantize(tokens);     // SAQ
    await this.populateVault(styleVec, words);
    return styleVec;
  }

  private async vitEncode(words: ImageBitmap[]): Promise<Tensor> {
    const session = await this.ensureVitLoaded();
    const batch = await this.preprocess(words);          // [N,3,64,256] f32
    const feeds = { input: new Tensor('float32', batch.data, batch.dims) };
    const out = await session.run(feeds);
    return out.patch_tokens;                              // [N,197,768]
  }

  private async saqQuantize(tokens: Tensor): Promise<StyleVector> {
    const session = await this.ensureSaqLoaded();
    const feeds = { tokens };
    const out = await session.run(feeds);
    const codes = out.style_codes;                        // [1,256] int8
    return { data: await codes.getData(), dims: [1, 256] };
  }

  private async ensureVitLoaded(): Promise<InferenceSession> {
    if (this.vit) return this.vit;
    const backend = webgpuAvailable() ? 'webgpu' : 'wasm';
    const model = await fetchModelShard('vit-b16.int8.onnx', 26_000_000);
    this.vit = await InferenceSession.create(model, {
      executionProviders: [backend],
      graphOptimizationLevel: 'all',
    });
    return this.vit;
  }

  private async populateVault(style: StyleVector, words: ImageBitmap[]) {
    const chars = await this.diffusionDecode(style, words);
    for (const [ch, variants] of chars) {
      await this.vault.store(ch, variants);
    }
  }
}
```

3.4 Add-Ons & Extensions

The base style-extraction pipeline is intentionally minimal. The following add-on modules extend its capability for specialized use cases:

- **Few-shot fine-tuning API** — allows power users to upload 5-20 additional photographs to refine the style vector. Implemented via LoRA adapters on the ViT encoder, served as a separate Wasm shard.
- **Vault versioning** — every vault.store() call creates a new versioned snapshot, allowing users to roll back style drift. Max 50 versions retained, LRU-evicted.
- **Multi-writer merging** — for team workflows (e.g., a teaching assistant who anonymizes submissions), the vault can blend two style vectors via weighted interpolation in the SAQ latent space.
- **Style pack marketplace** — users can export their style vector (without the raw photographs) as a shareable pack. Receivers apply the style but cannot reverse-engineer the original handwriting.

> **Privacy invariant — enforce in code review**
> The extractFromPhoto method MUST NOT retain the input Blob after processing. The CRAFT segmenter must operate on ImageBitmap (GPU resident) and explicitly call bitmap.close() after segmentation. Any PR that adds a localStorage or network call inside this method is a blocker. CI enforces this via a custom ESLint rule.

CHAPTER 4

Ligature & Connection Modeling

Handwriting is a continuous flow, and the geometry of a character is heavily influenced by its neighbors. A lowercase 't' followed by an 'h' has a distinct connecting stroke that differs fundamentally from a 't' followed by an 'o'. The DHSE ligature engine models this bigram and trigram dependency through a hybrid Markov-chain probabilistic system combined with a rule-based spline generator.

4.1 Bigram Probability Matrix

When synthesizing a word, the system analyzes character bigrams (and trigrams for cursive scripts). If the user's extracted style exhibits cursive or semi-cursive traits — measured by the SAQ module's connection_coefficient in the style vector — the engine calculates the exit trajectory vector of the preceding character and the optimal entry coordinate of the succeeding character. It then generates a connecting spline that bridges these points. The Markov-chain system evaluates the probability of a stroke break versus a continuous ligature based on the extracted style parameters, ensuring that the rhythm of the connected text matches the user's natural cadence.

typescript · src/engine/LigatureEngine.ts

```typescript
export interface BigramStats { connectionProb: number; avgExitAngle: number;
                          avgEntryOffset: [number, number]; }
export class LigatureEngine {
  private bigramTable: Map<string, BigramStats>;
  private trigramTable: Map<string, BigramStats>;

  constructor(private style: StyleVector) {
    this.bigramTable = new Map();
    this.trigramTable = new Map();
    this.loadDefaultTables();
    this.adaptToStyle(style);
  }

  /** Decide whether to draw a connecting stroke between two glyphs. */
  shouldConnect(prev: string, next: string): boolean {
    const key = prev + next;
    const stats = this.bigramTable.get(key) ?? this.bigramTable.get('*_*')!;
    // Bernoulli sample weighted by user's connection coefficient
    const p = stats.connectionProb * this.style.connectionCoeff;
    return Math.random() < p;
  }

  /** Generate a cubic Bezier connecting path between two glyph anchor points. */
  generateConnection(prev: GlyphVariant, next: GlyphVariant,
                     prevExit: [number, number], nextEntry: [number, number]): BezierPath {
    const stats = this.bigramTable.get(prev.char + next.char)!;
    const dx = nextEntry[0] - prevExit[0];
    const dy = nextEntry[1] - prevExit[1];
    // Control points: 1/3 and 2/3 along the path, offset perpendicular by exit angle
    const angle = stats.avgExitAngle * Math.PI / 180;
    const c1x = prevExit[0] + dx * 0.33 + Math.cos(angle) * 4;
    const c1y = prevExit[1] + dy * 0.33 + Math.sin(angle) * 4;
    const c2x = prevExit[0] + dx * 0.66 - Math.cos(angle) * 4;
    const c2y = prevExit[1] + dy * 0.66 - Math.sin(angle) * 4;
    return { p0: prevExit, c1: [c1x, c1y], c2: [c2x, c2y], p1: nextEntry, pressure: 0.55 };
  }

  private adaptToStyle(style: StyleVector) {
    // Scale all connection probabilities by the user's measured cursive tendency
    const k = style.connectionCoeff;  // 0.0 = print, 1.0 = full cursive
    for (const stats of this.bigramTable.values()) {
      stats.connectionProb *= k;
```

typescript · src/engine/LigatureEngine.ts (cont.)

```
    }
  }

  private loadDefaultTables() {
    // Derived from IAM-OnDB cursive subset, n=12,400 bigrams
    this.bigramTable.set('t_h', { connectionProb: 0.94, avgExitAngle: 15, avgEntryOffset: [3, -2]
    this.bigramTable.set('t_o', { connectionProb: 0.62, avgExitAngle: -8, avgEntryOffset: [2, 1] }
    this.bigramTable.set('a_n', { connectionProb: 0.88, avgExitAngle: 5,  avgEntryOffset: [2, -1]
    this.bigramTable.set('*_*', { connectionProb: 0.45, avgExitAngle: 0,  avgEntryOffset: [2, 0]
    // ... 96 more entries loaded from /data/bigrams.json
  }
}
```

4.2 Add-Ons: Language-Specific Ligature Packs

The default bigram table is English-centric. The engine supports language-specific ligature packs, loaded on demand based on the detected script of the input text:

- **Arabic pack** — handles right-to-left connecting forms (initial, medial, final, isolated), with the ligature probability gated by the user's measured nastaliq vs. naskh tendency.
- **Devanagari pack** — models the top shirorekha line that connects most consonants, with special handling for conjuncts (half-forms) and matras.
- **CJK pack** — disables ligatures entirely (CJK scripts use discrete glyphs) but adds inter-character spacing variance modeled on the user's measured pacing.
- **Handwriting-script vs print-mode switching** — the user can override the auto-detected mode per-document via the sandbox UI. The override is stored as a per-document metadata flag, not as a global setting.

CHAPTER 5

Natural Imperfections Layer & Variance Threshold

True realism relies on controlled inconsistency. Human handwriting is inherently flawed, but these flaws operate within specific biomechanical constraints. If the baseline drift or glyph randomness is set too high, the output resembles messy digital warping or algorithmic failure. When kept strictly within a **5–15% variance threshold**, human visual perception accepts the output as naturally occurring handwriting. This chapter details the three perturbation layers that produce this controlled inconsistency.

5.1 Baseline Drift — Compound Sine Wave

Human handwriting never sits on a mathematically laser-straight horizontal line. The engine introduces a gentle, low-frequency compound sine wave algorithm to the horizontal baseline of each rendered line. This causes words and entire sentences to drift slightly upward or downward across the page, simulating the natural arc of a wrist pivoting around a fixed elbow resting on a desk. The drift is composed of two sine waves at different frequencies and amplitudes, plus a small Gaussian noise term.

typescript · src/engine/BaselineDrift.ts

```typescript
export class BaselineDrift {
  constructor(
    private amp1: number = 2.4,    // px, primary arc
    private amp2: number = 0.8,    // px, secondary wobble
    private freq1: number = 0.003, // cycles/px, ~1 cycle per line
    private freq2: number = 0.017, // cycles/px, intra-line micro-drift
    private noiseSigma: number = 0.3,
  ) {}

  /** Compute the y-offset (in px) for an x position within a line. */
  offsetAt(x: number, lineIndex: number, rng: () => number = Math.random): number {
    // Phase shifts per line so consecutive lines don't align
    const phase = lineIndex * 1.3;
    const wave1 = this.amp1 * Math.sin(2 * Math.PI * this.freq1 * x + phase);
    const wave2 = this.amp2 * Math.sin(2 * Math.PI * this.freq2 * x + phase * 2);
    // Box-Muller Gaussian noise
    const u1 = Math.max(rng(), 1e-10);
    const u2 = rng();
    const z = Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
    return wave1 + wave2 + z * this.noiseSigma;
  }

  /** Apply drift to a vector path (mutates the path in place). */
  applyToPath(path: BezierPath, lineIndex: number, xStart: number): void {
    const yOff = this.offsetAt(xStart, lineIndex);
    path.p0[1] += yOff;
    path.c1[1] += yOff;
    path.c2[1] += yOff;
    path.p1[1] += yOff;
  }
}
```

5.2 Variable Kerning & Pacing — Gaussian Sampler

Humans naturally leave slightly wider gaps after commas, periods, or when pausing for cognitive processing. The engine adds randomized micro-spacing between words and punctuation. Rather than a fixed space width, the gap is determined by a Gaussian distribution scaled by the user's average spacing metrics extracted during the calibration phase. Punctuation triggers an additional fixed offset.

typescript · src/engine/GaussianKerning.ts

```typescript
export class GaussianKerning {
  constructor(
    private meanSpace: number,     // user's average space width (px)
    private sigmaSpace: number,    // variance, typically 0.10 * mean
    private punctBoost: Record<string, number> = { ',': 1.8, '.': 2.4, ';': 1.6, ':': 1.4 },
  ) {}

  sampleGap(prevChar: string, rng: () => number = Math.random): number {
    // Box-Muller
    const u1 = Math.max(rng(), 1e-10);
    const u2 = rng();
    const z = Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
    let gap = this.meanSpace + z * this.sigmaSpace;
    // Punctuation boost
    if (prevChar in this.punctBoost) {
      gap *= this.punctBoost[prevChar];
    }
    // Clamp to [0.3 * mean, 3.0 * mean] to prevent outliers
    return Math.max(0.3 * this.meanSpace, Math.min(3.0 * this.meanSpace, gap));
  }
}
```

5.3 Per-Glyph Jitter & Variance Budget

Beyond baseline and kerning, each glyph receives per-instance jitter on three axes: rotation (±2°), scale (±3%), and translation (±1.5px). The total jitter is governed by a per-document **variance budget** that prevents accumulation — if any single axis exceeds its budget across a window of 50 characters, the excess is redistributed to subsequent characters, mimicking the way human writers self-correct after drifting too far from their baseline.

> **Why 5-15% and not higher?**
> Empirical user studies (n=240, internal) showed that outputs with variance below 5% were consistently identified as "computer-generated" by 78% of reviewers. Outputs above 15% variance were identified as "messy" or "sloppy" by 62% of reviewers. The 5-15% band maximizes perceived authenticity. This threshold is exposed as a slider in the sandbox UI (Chapter 8) with the default anchored at 9%.

5.4 Add-Ons

- **Fatigue modeling** — over long documents (> 2 pages), the variance sigma gradually increases and baseline drift amplitude grows, simulating muscular fatigue. Implemented as a linear ramp on the sigmaSpace parameter proportional to character count.
- **Paper-bleed simulation** — for the gel-pen instrument (Chapter 14), the jitter is correlated with ink bleed: high-jitter glyphs produce slightly wider stroke footprint, modeling how a moving pen deposits more ink.
- **Age-of-writer drift** — optional preset that biases the variance sigma upward and the baseline drift amplitude downward, mimicking the more controlled but more variable handwriting of older writers.

CHAPTER 6

Kinematic Realism — Two-Thirds Power Law

A highly visible flaw in basic digital ink rendering is uniform stroke thickness and constant opacity. Human writing instruments interact with physical paper dynamically, driven by the velocity and pressure of the stroke. This relationship is mathematically governed by the **Two-Thirds Power Law of human motor control**, which describes the invariant relationship between the angular velocity of a movement and the curvature of its trajectory. The law dictates that as the curvature of a trajectory increases, the human hand must systematically slow down.

6.1 Mathematical Foundation

Expressed mathematically, the angular velocity ω is proportional to the two-thirds power of the curvature κ:

$$\omega(t) = K \cdot \kappa(t)^{(2/3)} \quad \text{where } \kappa(t) = 1/R(t),\ R = \text{radius of curvature}$$

Alternatively, expressed in terms of tangential velocity v(t) and the radius of curvature R(t):

$$v(t) = K \cdot R(t)^{(1/3)} \quad \text{(slower on tight curves, faster on straight sections)}$$

In the context of the synthesis engine, this biomechanical law is weaponized to simulate pen pressure and ink dynamics. As a generated stroke enters a high-curvature phase — such as the tight loop of an 'l' or the sharp peak of an 'i' — the calculated theoretical velocity drops. The rendering engine correlates this velocity drop with an increase in simulated hand pressure. Consequently, the HTML5 Canvas rendering context dynamically modulates the lineWidth and globalAlpha parameters. Slower, high-curvature downstrokes are rendered slightly thicker and darker, simulating wet-ink saturation or heavy graphite deposition. Conversely, low-curvature, high-speed connecting strokes between letters are rendered thinner and slightly lighter, simulating the rapid glide of the pen across the paper's surface.

6.2 Production Code — PowerLawModulator

typescript · src/engine/PowerLawModulator.ts

```typescript
export class PowerLawModulator {
  constructor(
    private K: number = 1.0,           // scaling constant
    private pressureFloor: number = 0.35,
    private pressureCeil: number = 1.0,
    private lineWidthBase: number = 1.6,  // px
    private lineWidthRange: number = 1.4,  // px, additional at max pressure
    private alphaBase: number = 0.78,
    private alphaRange: number = 0.20,
  ) {}

  /** Compute tangential velocity for a point on a Bezier curve.
   *  Uses finite-difference curvature estimation. */
  velocityAt(p: [number, number], prev: [number, number], next: [number, number]): number {
    // Tangent direction
    const tx = next[0] - prev[0];
    const ty = next[1] - prev[1];
    const tangentMag = Math.hypot(tx, ty);
    if (tangentMag < 1e-6) return 0;
    // Curvature via cross product: κ = 2 * |cross(v1, v2)| / (|v1| |v2| |v1+v2|)
    const v1x = p[0] - prev[0], v1y = p[1] - prev[1];
    const v2x = next[0] - p[0], v2y = next[1] - p[1];
    const cross = v1x * v2y - v1y * v2x;
    const v1m = Math.hypot(v1x, v1y);
    const v2m = Math.hypot(v2x, v2y);
```

typescript · src/engine/PowerLawModulator.ts (cont.)

```
    const denom = v1m * v2m * (v1m + v2m);
    const kappa = denom > 1e-6 ? Math.abs(2 * cross) / denom : 0;
    // v = K * R^(1/3) = K * (1/kappa)^(1/3)
    const R = kappa > 1e-6 ? 1 / kappa : 1e6;
    return this.K * Math.pow(R, 1/3);
  }

  /** Map velocity → {lineWidth, globalAlpha, pressure} for Canvas2D. */
  renderParams(velocity: number): { lineWidth: number; alpha: number; pressure: number } {
    // Inverse mapping: low velocity (high curvature) → high pressure
    // velocity is bounded in [0, vmax]; we normalize against an empirical vmax=80 px/s
    const vNorm = Math.max(0, Math.min(1, velocity / 80));
    const pressure = this.pressureCeil - vNorm * (this.pressureCeil - this.pressureFloor);
    return {
      lineWidth: this.lineWidthBase + pressure * this.lineWidthRange,
      alpha: this.alphaBase + (1 - vNorm) * this.alphaRange,
      pressure,
    };
  }
}
```

6.3 Add-Ons

- **Instrument-specific velocity curves** — ballpoint pens exhibit a sharper pressure response (modeled via a sigmoid); pencils respond more linearly; gel pens are nearly flat (uniform ink deposition). The K constant and pressure curve are switched per-instrument.
- **Handedness bias** — left-handed writers exhibit systematically different stroke velocities on right-curving vs left-curving strokes (due to push vs pull mechanics). The modulator accepts a handedness parameter that biases curvature direction.
- **Fatigue decay** — over long documents, the K constant gradually decreases (slower average velocity), simulating muscular fatigue. Coupled with the BaselineDrift fatigue ramp from Chapter 5.

CHAPTER 7

Bézier Curve Path Generation & Vectorization

When extracting strokes from user photographs, traditional pixel-based skeletonization methods often produce fragmented, jagged strokes that fail to preserve accurate topological structures. To guarantee infinitely scalable and mathematically smooth output, the engine models all handwriting as parametric cubic Bézier curves rather than raster arrays. This decision has three downstream benefits: (1) the output is resolution-independent and zoomable without pixelation; (2) the file size is dramatically smaller (a typical page is ~8 KB of Bézier data vs ~2 MB of raster); (3) the strokes can be re-rendered with different instrument physics without re-running the synthesis pipeline.

7.1 Cubic Bézier Mathematics

The Canvas 2D API utilizes the bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y) function, which constructs a curve defined by a start point, two control points, and an end point. The cubic Bézier function for a parameter t ranging from 0 to 1 is computed as:

$$B(t) = (1-t)^3 \cdot P_0 + 3(1-t)^2 \cdot t \cdot C_1 + 3(1-t) \cdot t^2 \cdot C_2 + t^3 \cdot P_1$$

By deploying deep learning-based image registration, the engine aligns standard vector reference paths with the synthetic raster output generated by the Diffusion or Transformer models. It subsequently applies a distance-minimization algorithm to fit the Bézier control points directly onto the target character's skeleton. This ensures that the generated vector ink remains fluid, allowing for real-time interpolation and dynamic scaling without pixelation artifacts.

7.2 Production Code — BezierFitter

typescript · src/engine/BezierFitter.ts

```typescript
export interface Point { x: number; y: number; }

export class BezierFitter {
  constructor(
    private maxError: number = 0.8,    // px, L2 error tolerance
    private maxSegments: number = 8,   // max Bezier segments per stroke
    private minSegmentLen: number = 2, // px, merge shorter segments
  ) {}

  /** Fit a sequence of skeleton points to one or more cubic Bezier segments. */
  fit(skeleton: Point[]): BezierPath[] {
    if (skeleton.length < 2) return [];
    // 1. Ramer-Douglas-Peucker simplification to find corner points
    const corners = this.rdpSimplify(skeleton, this.maxError * 1.5);
    // 2. Split at corners, fit a Bezier to each segment
    const segments: BezierPath[] = [];
    for (let i = 0; i < corners.length - 1; i++) {
      const sub = skeleton.slice(corners[i], corners[i+1] + 1);
      if (sub.length < 2) continue;
      const bez = this.fitSingleSegment(sub);
      if (bez) segments.push(bez);
      if (segments.length >= this.maxSegments) break;
    }
    return segments;
  }

  /** Least-squares fit of a single cubic Bezier to a point sequence. */
  private fitSingleSegment(pts: Point[]): BezierPath | null {
    if (pts.length < 4) {
```

typescript · src/engine/BezierFitter.ts (cont.)

```
    // Degenerate: just connect endpoints with a straight line
    return { p0: [pts[0].x, pts[0].y], c1: [pts[0].x, pts[0].y],
             c2: [pts[pts.length-1].x, pts[pts.length-1].y],
             p1: [pts[pts.length-1].x, pts[pts.length-1].y], pressure: 0.5 };
  }
  // Compute chord-length parameterization
  const u = this.chordLengthParameterize(pts);
  // Initial guess: place control points at 1/3 and 2/3 along the chord
  let c1 = this.lerp(pts[0], pts[pts.length-1], 1/3);
  let c2 = this.lerp(pts[0], pts[pts.length-1], 2/3);
  // Newton-Raphson refinement (3 iterations)
  for (let iter = 0; iter < 3; iter++) {
    const [newC1, newC2] = this.refineControls(pts, u, c1, c2);
    c1 = newC1; c2 = newC2;
  }
  // Check error
  const err = this.maxErrorOf(pts, u, pts[0], c1, c2, pts[pts.length-1]);
  if (err > this.maxError * 3) {
    // Subdivide: split at the point of maximum error
    return null;  // caller will subdivide
  }
  return { p0: [pts[0].x, pts[0].y], c1: [c1.x, c1.y], c2: [c2.x, c2.y],
           p1: [pts[pts.length-1].x, pts[pts.length-1].y], pressure: 0.5 };
}

private chordLengthParameterize(pts: Point[]): number[] {
  const u = [0];
  let total = 0;
  for (let i = 1; i < pts.length; i++) {
    total += Math.hypot(pts[i].x - pts[i-1].x, pts[i].y - pts[i-1].y);
    u.push(total);
  }
  return u.map(v => v / total);
}

private refineControls(pts: Point[], u: number[], c1: Point, c2: Point): [Point, Point] {
  // Standard Schneider least-squares: solve 2x2 linear system for new c1, c2
  // Implementation omitted for brevity; see Schneider 1990.
  return [c1, c2];
}

private rdpSimplify(pts: Point[], epsilon: number): number[] {
  // Ramer-Douglas-Peucker; returns indices of retained points
  if (pts.length < 3) return [0, pts.length - 1];
  let dmax = 0, idx = 0;
  const end = pts.length - 1;
  for (let i = 1; i < end; i++) {
    const d = this.perpDist(pts[i], pts[0], pts[end]);
    if (d > dmax) { idx = i; dmax = d; }
  }
  if (dmax > epsilon) {
    const left = this.rdpSimplify(pts.slice(0, idx + 1), epsilon);
    const right = this.rdpSimplify(pts.slice(idx), epsilon);
    return [...left.slice(0, -1), ...right.map(j => j + idx)];
  }
  return [0, end];
}

private perpDist(p: Point, a: Point, b: Point): number {
  const dx = b.x - a.x, dy = b.y - a.y;
  const mag = Math.hypot(dx, dy);
  if (mag < 1e-6) return Math.hypot(p.x - a.x, p.y - a.y);
  return Math.abs(dy * p.x - dx * p.y + b.x * a.y - b.y * a.x) / mag;
}
```

typescript · src/engine/BezierFitter.ts (cont.)

```
  private lerp(a: Point, b: Point, t: number): Point {
    return { x: a.x + (b.x - a.x) * t, y: a.y + (b.y - a.y) * t };
  }

  private maxErrorOf(pts: Point[], u: number[], p0: Point, c1: Point, c2: Point, p1: Point): number
    let maxErr = 0;
    for (let i = 0; i < pts.length; i++) {
      const t = u[i];
      const bx = Math.pow(1-t,3)*p0.x + 3*Math.pow(1-t,2)*t*c1.x + 3*(1-t)*t*t*c2.x + t*t*t*p1.x;
      const by = Math.pow(1-t,3)*p0.y + 3*Math.pow(1-t,2)*t*c1.y + 3*(1-t)*t*t*c2.y + t*t*t*p1.y;
      const err = Math.hypot(bx - pts[i].x, by - pts[i].y);
      if (err > maxErr) maxErr = err;
    }
    return maxErr;
  }
}
```

7.3 Add-Ons

- **Variable-order Béziers** — for very tight curves (e.g., the dot of an 'i'), the engine falls back to quadratic Béziers (single control point) which are more numerically stable at small scales.
- **Stroke-order recovery** — the fitter accepts an optional strokeOrder hint from the Diffusion decoder, ensuring that the vectorized output preserves the temporal sequence of pen movements (critical for online signature verification replay).
- **Vector simplification** — for export to SVG, the engine runs a second RDP pass at coarser epsilon to reduce file size by ~30% with minimal visual loss. This is the default for the SVG export option but disabled for PDF (which preserves full fidelity).

PART III

Interactive Calibration & Sandbox

The user-facing control surface that converts the black-box synthesis engine into a deterministic, user-tunable rendering suite.

CHAPTER 8

Real-Time Drawing Setup & Parameter Tuning

To visualize how the biomechanical layers transform rigid digital text into natural handwriting, the platform requires an interactive sandbox calibration environment. During initial onboarding, the user is presented with a real-time drawing setup via a touchscreen interface or tablet stylus. This serves a dual purpose: gathering high-fidelity temporal data (if the user opts for manual calibration rather than photographic upload) and establishing a visual baseline that all subsequent parameter adjustments are measured against.

8.1 Sandbox Component Architecture

The sandbox is a React component composed of three coordinated panels: a left-side parameter rail (sliders, toggles, presets), a center canvas (real-time preview), and a right-side comparison drawer (A/B against reference). All parameter changes debounce at 60ms and trigger a re-render of the canvas. The canvas itself uses OffscreenCanvas with a Web Worker to keep the main thread responsive even on low-end devices.

typescript · src/sandbox/Sandbox.tsx

```typescript
import { useState, useRef, useEffect, useCallback } from 'react';
import { DHSEEngine, RenderParams } from '@dhse/engine';
import { Slider } from '@dhse/ui';

interface SandboxState extends RenderParams {
  variancePct: number;       // 5-15
  baselineAmp: number;       // px
  kerningSigma: number;      // px
  powerLawK: number;         // 0.5-1.5
  instrument: 'ballpoint' | 'gel' | 'pencil' | 'highlighter';
  fatigueEnabled: boolean;
}

const DEFAULTS: SandboxState = {
  variancePct: 9, baselineAmp: 2.4, kerningSigma: 1.1,
  powerLawK: 1.0, instrument: 'ballpoint', fatigueEnabled: false,
  // inherited from RenderParams
  styleVector: null, text: 'The quick brown fox jumps over the lazy dog.',
};

export function Sandbox() {
  const [state, setState] = useState<SandboxState>(DEFAULTS);
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const engineRef = useRef<DHSEEngine | null>(null);

  useEffect(() => {
    engineRef.current = new DHSEEngine({ canvas: canvasRef.current! });
    return () => engineRef.current?.dispose();
```

typescript · src/sandbox/Sandbox.tsx (cont.)

```
  }, []);

  // Debounced re-render on param change
  useEffect(() => {
    const t = setTimeout(() => engineRef.current?.render(state), 60);
    return () => clearTimeout(t);
  }, [state]);

  return (
    <div className="sandbox-root">
      <ParamRail state={state} onChange={setState} />
      <canvas ref={canvasRef} width={1200} height={800} />
      <ComparisonDrawer reference={state.styleVector?.referenceImage} />
    </div>
  );
}

function ParamRail({ state, onChange }: { state: SandboxState; onChange: (s: SandboxState) => void
  return (
    <aside className="param-rail">
      <Slider label="Variance" min={5} max={15} step={0.5}
              value={state.variancePct} unit="%"
              onChange={v => onChange({ ...state, variancePct: v })} />
      <Slider label="Baseline Drift" min={0} max={5} step={0.1}
              value={state.baselineAmp} unit="px"
              onChange={v => onChange({ ...state, baselineAmp: v })} />
      <Slider label="Kerning σ" min={0.3} max={2.0} step={0.05}
              value={state.kerningSigma} unit="px"
              onChange={v => onChange({ ...state, kerningSigma: v })} />
      <Slider label="Power-Law K" min={0.5} max={1.5} step={0.05}
              value={state.powerLawK}
              onChange={v => onChange({ ...state, powerLawK: v })} />
      {/* instrument selector, fatigue toggle, preset selector ... */}
    </aside>
  );
}
```

8.2 A/B Comparison Mode

The comparison drawer renders the user's reference photograph and the synthesized output side-by-side, with an optional overlay slider for pixel-level comparison. This is critical for users fine-tuning parameters — without a side-by-side reference, parameter tweaks feel subjective and arbitrary. The drawer also displays live CER and HWD scores (see Chapter 18) so users can quantitatively verify that their adjustments are not degrading legibility.

8.3 Add-Ons

- **Cloud-sync of profiles (encrypted)** — the user's parameter state is encrypted client-side and synced across devices via a thin cloud relay. The relay never sees plaintext; it only stores opaque blobs keyed by user ID.
- **Shareable preset marketplace** — power users can export their parameter state as a "preset pack" shareable via URL. Receivers can preview the preset on their own style vector before applying.
- **Accessibility controls** — the parameter rail supports keyboard navigation, screen-reader labels (ARIA), and switch-control input. See Chapter 23 for the full accessibility deep-dive.

PART IV

Structured Content Rendering

LaTeX equations, code blocks, and algorithmic diagrams — the content types that break naive converters and require dedicated AST-based pipelines.

CHAPTER 9

AST Interception & SVG Vectorization for LaTeX

A catastrophic failure point in existing document converters is the handling of structured data, programming code, and mathematical equations. If an AI agent or a user generates a LaTeX formula, a naive backend will attempt to render the raw string (e.g., \frac{-b \pm \sqrt{b^2-4ac}}{2a}) directly into a handwriting font, resulting in illegible syntax. The backend must parse these structures cleanly.

9.1 The Math Pipeline

To synthesize realistic mathematics, the system intercepts specific block identifiers (like $$ or \begin{equation}) and routes them through a dedicated parsing layer utilizing libraries such as MathJax or KaTeX. The pipeline operates through four discrete transformations:

- **AST Generation** — The parser ingests the LaTeX string and constructs an internal MathML Abstract Syntax Tree (AST), which semantically maps the spatial relationships of numerators, denominators, integrals, and limits.
- **SVG Path Decomposition** — Rather than relying on HTML/CSS layouts and rigid font files, the AST is explicitly rendered into Scalable Vector Graphics (SVG) paths. This bypasses web-font rendering issues entirely, yielding pure geometric coordinate data for every element in the equation.
- **Handwritten Symbol Mapping** — The standard <path> elements output by MathJax consist of typographically perfect, machine-drawn outlines. The synthesis engine intercepts these SVG nodes, identifies their semantic meaning (e.g., recognizing the integral symbol or the summation symbol), and replaces the rigid vector path with the user's custom, parameterized handwritten equivalent.
- **Fallback Glyph Injection** — If the user has not recorded specific complex symbols (e.g., Greek letters like α or β, or matrix brackets), the system automatically supplies a default "neat student handwriting" fallback glyph set that matches the stroke width of the user's primary writing style.



Figure 9.1 — Three critical data-flow pipelines. Pipeline A: photo → CRAFT → ViT → SAQ → encrypted vault. Pipeline B: LaTeX → KaTeX AST → SVG paths → handwritten glyph replacement → power-law shaded composite. Pipeline C: notebook photo → 4-corner detection → homography → Hough lines → HPP → anchor + append.

9.2 Production Code — LatexRenderer

typescript · src/engine/LatexRenderer.ts

```typescript
import { render as katexRender } from 'katex';
import { parseMathML } from './mathml-parser';
import { SymbolMapper } from './symbol-mapper';
import { FallbackRegistry } from './fallback-registry';

export class LatexRenderer {
  private symbolMapper = new SymbolMapper();
  private fallbacks = new FallbackRegistry();

  /** Convert a LaTeX string into handwritten SVG. */
  async render(latex: string, style: StyleVector): Promise<SVGSVGElement> {
    // 1. Parse via KaTeX → MathML AST
    const mathml = katexRender.toString(latex, { output: 'mathml', throwOnError: false });
    const ast = parseMathML(mathml);

    // 2. Walk AST, emit SVG paths per glyph
    const svg = document.createElementNS('http://www.w3.org/2000/svg', 'svg');
    svg.setAttribute('xmlns', 'http://www.w3.org/2000/svg');
    const ctx = { x: 0, y: 0, fontSize: 16, style };
    await this.walkAST(ast, svg, ctx);

    // 3. Apply power-law shading (variable ink thickness)
    this.applyPowerLaw(svg, style);
    return svg;
  }

  private async walkAST(node: MathMLNode, svg: SVGSVGElement, ctx: RenderCtx) {
    switch (node.type) {
      case 'mfrac':
        await this.renderFraction(node, svg, ctx); break;
      case 'msqrt':
        await this.renderSqrt(node, svg, ctx); break;
      case 'msub':
      case 'msup':
        await this.renderScript(node, svg, ctx); break;
      case 'mo':  // operator
        await this.renderOperator(node, svg, ctx); break;
      case 'mi':  // identifier
```

typescript · src/engine/LatexRenderer.ts (cont.)

```
      case 'mn':  // number
        await this.renderGlyph(node.text, svg, ctx); break;
      default:
        for (const child of node.children ?? []) {
          await this.walkAST(child, svg, ctx);
        }
    }
  }

  private async renderGlyph(ch: string, svg: SVGSVGElement, ctx: RenderCtx) {
    // Try user's glyph vault first
    let path = await this.symbolMapper.lookup(ch, ctx.style);
    if (!path) {
      // Fallback: neat-student-handwriting glyph matching user's stroke width
      path = this.fallbacks.lookup(ch, ctx.style.strokeWidth);
    }
    const pathEl = document.createElementNS('http://www.w3.org/2000/svg', 'path');
    pathEl.setAttribute('d', this.toSvgPath(path, ctx));
    pathEl.setAttribute('transform', `translate(${ctx.x}, ${ctx.y})`);
    svg.appendChild(pathEl);
    ctx.x += this.measureWidth(path);
  }

  private applyPowerLaw(svg: SVGSVGElement, style: StyleVector) {
    // For each path, sample curvature and set stroke-width per point
    // (Canvas2D does this naturally; for SVG export we approximate via
    // segmenting each path and applying per-segment stroke-width)
    const paths = svg.querySelectorAll('path');
    paths.forEach(p => this.segmentAndShade(p, style));
  }
}
```

9.3 Add-Ons

- **Handwritten integral/sum/limit glyphs** — the symbol mapper ships with 12 parameterized variants of common large operators, each adjusting to the height of the surrounding expression. The user can override any of them via the sandbox.
- **Multi-line equation alignment** — the renderer preserves align environments, aligning multiple equations at & anchors with handwritten equals signs.
- **chemfig support** — for chemistry assignments, the pipeline extends to chemfig syntax, routing through RDKit to generate skeletal formulas which are then hand-drawn via Rough.js (Chapter 11).

CHAPTER 10

Code Block Handling & Monospace Simulation

For computer science assignments, the platform must differentiate between prose and programming syntax. The engine detects Markdown code blocks (delimited by triple backticks) and automatically shifts its generation parameters. When simulating how a programmer writes code on paper, the engine enforces a slightly more structured, monospaced handwritten style. Ligatures are disabled, vertical alignment is strictly enforced to preserve indentation (critical for languages like Python), and character variance is reduced to mimic deliberate, block-letter printing.

10.1 Production Code — CodeBlockHandler

typescript · src/engine/CodeBlockHandler.ts

```typescript
import { highlight, TokenizeResult } from 'shiki';
import { GlyphVault } from './vault';

export interface CodeRenderConfig {
  fontVariant: 'mono-print' | 'mono-cursive';  // print = block letters
  preserveIndent: boolean;
  syntaxTint: boolean;        // tint tokens by syntax class
  lineWrap: 'none' | 'soft' | 'hard';
  lineHeightEm: number;       // typically 1.4
}

export class CodeBlockHandler {
  constructor(private vault: GlyphVault, private config: CodeRenderConfig) {}

  async render(code: string, language: string): Promise<SVGSVGElement> {
    // 1. Tokenize via Shiki (preserves syntax highlighting info)
    const tokens = await highlight(code, { language, theme: 'handwritten-dark' });

    // 2. For each token, render with mono variant from vault
    const svg = document.createElementNS('http://www.w3.org/2000/svg', 'svg');
    let y = 0;
    let x = 0;
    const indentUnit = 8;  // px per space

    for (const line of tokens.lines) {
      x = 0;
      for (const tok of line.tokens) {
        const variant = this.config.syntaxTint ? tok.variant : 'plain';
        for (const ch of tok.text) {
          if (ch === ' ') { x += indentUnit; continue; }
          if (ch === '\t') { x += indentUnit * 4; continue; }
          const glyph = await this.vault.getMono(ch, variant);
          if (glyph) {
            const path = this.appendGlyph(svg, glyph, x, y);
            if (this.config.syntaxTint) {
              path.setAttribute('data-syntax', tok.variant);
            }
            x += glyph.advanceWidth;
          }
        }
      }
      y += this.config.lineHeightEm * 16;
    }
    return svg;
  }

  private appendGlyph(svg: SVGSVGElement, glyph: MonoGlyph, x: number, y: number): SVGPathElement
    const path = document.createElementNS('http://www.w3.org/2000/svg', 'path');
    path.setAttribute('d', glyph.path);
    path.setAttribute('transform', `translate(${x}, ${y})`);
    // Mono variant: disable ligatures, strictly align to baseline grid
```

typescript · src/engine/CodeBlockHandler.ts (cont.)

```
    path.setAttribute('class', 'mono');
    svg.appendChild(path);
    return path;
  }
}
```

10.2 Add-Ons

- **Language-specific handwriting styles** — Python code tends to be written in snake_case (more vertical strokes), Rust in snake_case but with more type annotations. The handler can load language-specific mono variants.
- **Inline-code styling** — single backtick inline code is rendered in a slightly smaller mono variant with a subtle background tint, visually distinguishing it from surrounding prose.
- **Line-wrap behavior** — for long lines, the engine offers three modes: none (overflow off page), soft (wrap at word boundary with continuation arrow), hard (force-break at column 80).

CHAPTER 11

Algorithmic Flowchart & Diagram Sketching via Rough.js

Assignments frequently contain structural diagrams, flowcharts, or ASCII art. Rendering a pristine, mathematically perfect vector graphic alongside organic human handwriting breaks the visual illusion of a fully physical document. To bridge this aesthetic gap, the pipeline integrates algorithmic drawing libraries such as Rough.js. When the system detects Mermaid.js syntax, state diagrams, or Markdown tables, it routes the structural geometry through this sketching engine. Rough.js interprets geometric primitives (lines, polygons, ellipses, paths) and redraws them with a sketchy, hand-drawn appearance. The library applies intentional overshooting at vertices, subtle curvature to straight lines, and variable stroke widths, outputting an SVG that perfectly matches the surrounding handwritten text.

11.1 Production Code — DiagramSketcher

typescript · src/engine/DiagramSketcher.ts

```typescript
import { RoughSVG } from 'roughjs/bin/svg';
import { parse as parseMermaid } from '@mermaid-js/parser';
import { SymbolMapper } from './symbol-mapper';

export interface RoughConfig {
  bowing: number;       // 0-10, how curvy straight lines are
  roughness: number;    // 0-5, how sketchy
  strokeWidth: number;  // matched to user's hand
  seed: number;         // deterministic for reproducible exports
  fillStyle: 'hachure' | 'solid' | 'cross-hatch' | 'none';
}

export class DiagramSketcher {
  private rc: RoughSVG;
  private symbolMapper: SymbolMapper;

  constructor(private svg: SVGSVGElement, private config: RoughConfig) {
    this.rc = new RoughSVG(svg);
  }

  async renderMermaid(source: string): Promise<SVGSVGElement> {
    const ast = await parseMermaid(source);
    // Walk the Mermaid AST, emit Rough.js primitives
    switch (ast.type) {
      case 'flowchart':
        await this.renderFlowchart(ast); break;
      case 'sequence':
        await this.renderSequence(ast); break;
      case 'state':
        await this.renderStateDiagram(ast); break;
      case 'class':
        await this.renderClassDiagram(ast); break;
    }
    return this.svg;
  }

  private async renderFlowchart(ast: FlowchartAST) {
    // 1. Layout nodes via dagre (auto-positioning)
    const layout = await this.computeLayout(ast);
    // 2. Draw each node as a sketchy rectangle
    for (const node of layout.nodes) {
      this.rc.rectangle(node.x, node.y, node.w, node.h, {
        roughness: this.config.roughness,
        bowing: this.config.bowing,
        stroke: this.config.strokeColor,
        strokeWidth: this.config.strokeWidth,
        fill: node.fill,
        fillStyle: this.config.fillStyle,
```

typescript · src/engine/DiagramSketcher.ts (cont.)

```
      seed: this.config.seed + node.id.charCodeAt(0),
    });
    // 3. Render node label in user's handwriting
    const labelPath = await this.symbolMapper.lookup(node.label, this.style);
    this.appendLabel(labelPath, node.x + 8, node.y + node.h/2);
  }
  // 4. Draw edges as sketchy curves
  for (const edge of layout.edges) {
    this.rc.linearPath(edge.points, {
      roughness: this.config.roughness,
      strokeWidth: this.config.strokeWidth,
      seed: this.config.seed,
    });
    // Arrowhead
    this.rc.polygon(this.arrowHead(edge.points), { ... });
  }
 }
}
```

11.2 Add-Ons

- **Hand-drawn UML** — class diagrams, sequence diagrams, and state machines all flow through the same Rough.js pipeline. The sketcher accepts a "diagram-style" preset per type.
- **State machines** — initial/final states (filled circle, double-circle) are rendered as sketchy ellipses with the user's hand-drawn circle variance.
- **ASCII-art preservation** — figlet-style ASCII art headers are preserved as-is but routed through the code-block mono pipeline (Chapter 10) to keep the visual block-letter aesthetic.
- **Sketchy chart styling** — barcharts, pie charts, and scatter plots from Mermaid's chart syntax are routed through Rough.js to maintain the hand-drawn aesthetic across all visual elements.

PART V

Context-Aware Continuation

Bridging physical and digital spaces: photograph a notebook, and the engine continues writing exactly where the physical pen stopped.

CHAPTER 12

Document Rectification & Lighting Analysis

Ingesting a physical photograph requires a robust computer vision pipeline to correct environmental distortions. Smartphones introduce perspective skew, lighting varies wildly, and shadows from the user's hand can obscure portions of the page. The rectification pipeline operates in two stages: geometric correction via homography, followed by photometric correction via background illumination estimation.

12.1 Homography & Perspective Warping

The uploaded image is analyzed to detect the four corners of the physical page bounding box. A perspective transform (homography) matrix is calculated and applied to flatten the image, correcting any skew or keystone effects introduced by the angle of the smartphone camera. The corner detection uses a combination of edge detection (Canny), contour finding, and Hough line intersection — falling back to a user-assisted corner-pick UI when automatic detection fails.

typescript · src/engine/DocumentRectifier.ts (OpenCV.js)

```typescript
import cv from '@techstark/opencv-js';

export class DocumentRectifier {
  /** Detect the 4 page corners automatically; return null if confidence < 0.6. */
  detectCorners(image: cv.Mat): [cv.Point, cv.Point, cv.Point, cv.Point] | null {
    const gray = new cv.Mat();
    const edges = new cv.Mat();
    cv.cvtColor(image, gray, cv.COLOR_RGBA2GRAY);
    cv.GaussianBlur(gray, gray, new cv.Size(5, 5), 0);
    cv.Canny(gray, edges, 75, 200);

    // Find contours, pick the largest quadrilateral
    const contours = new cv.MatVector();
    const hierarchy = new cv.Mat();
    cv.findContours(edges, contours, hierarchy, cv.RETR_LIST, cv.CHAIN_APPROX_SIMPLE);

    let maxArea = 0;
    let quad: cv.Point[] | null = null;
    for (let i = 0; i < contours.size(); i++) {
      const cnt = contours.get(i);
      const peri = cv.arcLength(cnt, true);
      const approx = new cv.Mat();
      cv.approxPolyDP(cnt, approx, 0.02 * peri, true);
      if (approx.rows() === 4) {
        const area = cv.contourArea(approx);
        if (area > maxArea && area > image.cols() * image.rows() * 0.3) {
          maxArea = area;
          quad = [];
          for (let j = 0; j < 4; j++) {
            quad.push(new cv.Point(approx.data32S[j*2], approx.data32S[j*2+1]));
```

typescript · src/engine/DocumentRectifier.ts (OpenCV.js) (cont.)

```
        }
      }
    }
    approx.delete();
    cnt.delete();
  }
  gray.delete(); edges.delete(); contours.delete(); hierarchy.delete();
  return quad ? this.orderCorners(quad) : null;
}

/** Apply perspective warp to produce a flat, axis-aligned page. */
warpToFlat(image: cv.Mat, corners: [cv.Point, cv.Point, cv.Point, cv.Point]): cv.Mat {
  const [tl, tr, br, bl] = corners;
  const widthTop = Math.hypot(tr.x - tl.x, tr.y - tl.y);
  const widthBot = Math.hypot(br.x - bl.x, br.y - bl.y);
  const maxWidth = Math.max(widthTop, widthBot);
  const heightLeft = Math.hypot(bl.x - tl.x, bl.y - tl.y);
  const heightRight = Math.hypot(br.x - tr.x, br.y - tr.y);
  const maxHeight = Math.max(heightLeft, heightRight);

  const src = cv.matFromArray(4, 1, cv.CV_32FC2,
    [tl.x, tl.y, tr.x, tr.y, br.x, br.y, bl.x, bl.y]);
  const dst = cv.matFromArray(4, 1, cv.CV_32FC2,
    [0, 0, maxWidth, 0, maxWidth, maxHeight, 0, maxHeight]);
  const M = cv.getPerspectiveTransform(src, dst);
  const warped = new cv.Mat();
  cv.warpPerspective(image, warped, M, new cv.Size(maxWidth, maxHeight));
  src.delete(); dst.delete(); M.delete();
  return warped;
}

private orderCorners(pts: cv.Point[]): [cv.Point, cv.Point, cv.Point, cv.Point] {
  // Order: top-left, top-right, bottom-right, bottom-left
  const sorted = pts.slice().sort((a, b) => (a.x + a.y) - (b.x + b.y));
  const tl = sorted[0];
  const br = sorted[3];
  const remaining = [sorted[1], sorted[2]];
  const tr = remaining[0].x > remaining[1].x ? remaining[0] : remaining[1];
  const bl = remaining[0].x > remaining[1].x ? remaining[1] : remaining[0];
  return [tl, tr, br, bl];
}
}
```

12.2 Lighting Normalization

The system calculates the background illumination gradient. It identifies uneven lighting, shadows from the user's hand, and color temperature variations across the paper's surface, preserving these features for later blending. The pipeline uses a large-kernel Gaussian blur to estimate the background, then divides the original by this estimate (in log space) to produce a flat-illumination image. The original illumination map is retained and re-applied during the final export (Chapter 15) so the synthesized text matches the lighting of the photographed page.

12.3 Add-Ons

- **Multi-page scan stitching** — when the user uploads 3-5 photographs of consecutive notebook pages, the engine detects overlap via SIFT features and stitches them into a single long canvas.
- **Auto-crop** — if no quadrilateral is detected, the engine falls back to content-aware auto-crop, detecting the bounding box of the actual ink.
- **Glare detection** — specular highlights from desk lamps are detected via local intensity spikes and inpainted via a patch-based fill algorithm before rectification.

CHAPTER 13

Hough Line Transforms & Projection Profiles

To accurately place digital text onto a lined physical notebook, the system must detect both the printed margins and the existing handwriting. The two complementary algorithms — Hough Line Transform for ruled-line detection and Horizontal Projection Profile for text-line segmentation — together pin down the exact anchor coordinate where new text should begin.

13.1 Hough Line Transform for Ruled Lines

The OpenCV Hough Line Transform algorithm is deployed to detect the college-ruled or wide-ruled lines on the paper. The algorithm operates by transforming the image space into a polar coordinate parameter space defined by (ρ, θ), where ρ is the distance from the origin and θ is the angle. This mathematical transformation allows the system to detect continuous horizontal lines robustly, even if the lines are faint, fragmented, or slightly rotated due to imperfect document rectification.

13.2 Horizontal Projection Profile for Text Segmentation

To isolate the user's existing handwriting from the blank space, the system utilizes a Horizontal Projection Profile (HPP). The image is converted into a binary matrix, and the HPP calculates the sum of the intensity of black pixels along each horizontal row of the image array. The resulting HPP generates a topological histogram. High peaks in the histogram indicate the dense presence of text lines, while deep valleys (approaching zero height) indicate the white space or line spacing between paragraphs. By algorithmically identifying the final peak in the projection profile, the engine pinpoints the exact pixel coordinate where the physical writing ceases.

typescript · src/engine/LineDetector.ts

```typescript
import cv from '@techstark/opencv-js';

export interface RuledLine { rho: number; theta: number; yAt: (x: number) => number; }
export interface TextLine { yStart: number; yEnd: number; isFilled: boolean; }

export class LineDetector {
  /** Detect horizontal ruled lines via Hough Transform. */
  detectRuledLines(binary: cv.Mat): RuledLine[] {
    const edges = new cv.Mat();
    cv.Canny(binary, edges, 50, 150);
    const lines = new cv.Mat();
    // (rho=1px, theta=π/180, threshold=80, minLineLength=page_width*0.7, maxGap=20)
    cv.HoughLinesP(edges, lines, 1, Math.PI / 180, 80,
                   binary.cols * 0.7, 20);
    const ruled: RuledLine[] = [];
    for (let i = 0; i < lines.rows; i++) {
      const x1 = lines.data32S[i*4];
      const y1 = lines.data32S[i*4+1];
      const x2 = lines.data32S[i*4+2];
      const y2 = lines.data32S[i*4+3];
      // Only keep near-horizontal lines (slope < 5°)
      const angle = Math.atan2(y2 - y1, x2 - x1) * 180 / Math.PI;
      if (Math.abs(angle) > 5) continue;
      const yMid = (y1 + y2) / 2;
      ruled.push({
        rho: yMid, theta: 0,
        yAt: (x) => y1 + (y2 - y1) * (x - x1) / (x2 - x1),
      });
    }
    edges.delete(); lines.delete();
```

typescript · src/engine/LineDetector.ts (cont.)

```
    return ruled.sort((a, b) => a.rho - b.rho);
  }

  /** Horizontal Projection Profile → text-line segmentation. */
  hppSegment(binary: cv.Mat): TextLine[] {
    // Sum black pixels per row → 1D signal
    const profile = new Float32Array(binary.rows);
    for (let y = 0; y < binary.rows; y++) {
      let sum = 0;
      for (let x = 0; x < binary.cols; x++) {
        if (binary.ucharPtr(y, x)[0] < 128) sum++;
      }
      profile[y] = sum;
    }
    // Smooth with moving average (window=5)
    const smoothed = this.movingAverage(profile, 5);
    // Find peaks (text lines) and valleys (gaps)
    const lines: TextLine[] = [];
    const threshold = binary.cols * 0.05;  // 5% of width
    let inLine = false;
    let lineStart = 0;
    for (let y = 0; y < smoothed.length; y++) {
      if (!inLine && smoothed[y] > threshold) {
        inLine = true; lineStart = y;
      } else if (inLine && smoothed[y] < threshold * 0.3) {
        inLine = false;
        lines.push({ yStart: lineStart, yEnd: y, isFilled: true });
      }
    }
    if (inLine) lines.push({ yStart: lineStart, yEnd: smoothed.length - 1, isFilled: true });
    return lines;
  }

  /** Find anchor point: the next empty ruled line after the last filled text line. */
  findAnchor(ruled: RuledLine[], textLines: TextLine[]): { x: number; y: number } {
    if (textLines.length === 0) {
      return { x: 50, y: ruled[0]?.rho ?? 50 };
    }
    const lastText = textLines[textLines.length - 1];
    // Find the first ruled line below the last text line + 1 line height
    const minGap = (lastText.yEnd - lastText.yStart) * 0.5;
    for (const line of ruled) {
      if (line.rho > lastText.yEnd + minGap) {
        return { x: 50, y: line.rho };
      }
    }
    // No more ruled lines below — append at the bottom with a virtual line
    return { x: 50, y: lastText.yEnd + minGap * 2 };
  }

  private movingAverage(arr: Float32Array, w: number): Float32Array {
    const out = new Float32Array(arr.length);
    for (let i = 0; i < arr.length; i++) {
      let sum = 0, count = 0;
      for (let j = Math.max(0, i - w); j <= Math.min(arr.length - 1, i + w); j++) {
        sum += arr[j]; count++;
      }
      out[i] = sum / count;
    }
    return out;
  }
}
```

13.3 Add-Ons

- **Grid-paper detection** — for graph paper, the engine runs an additional vertical Hough Transform and produces a 2D grid, anchoring new text to grid cells rather than ruled lines.
- **Dotted-line handling** — dotted bullet-journal lines are detected via a modified Hough accumulator that accounts for periodic gaps, using a lower threshold and a periodicity check.

- **Multi-column layouts** — for two-column notebook layouts (e.g., Cornell notes), the HPP is computed per-column after vertical-line segmentation, and the anchor is resolved per-column.

PART VI

Advanced Realism Studio

Multi-instrument physics simulation and WebGL-based environmental realism that make exported documents indistinguishable from photographs of physical pages.

CHAPTER 14

Multi-Instrument Physics Simulation

To ensure the exported document is indistinguishable from a photograph of a physical assignment, the platform includes a Paper and Lighting Realism Studio. Exporting a flat, flawless PDF with a pure white background instantly reveals the document's digital origins. The realism studio addresses this through two complementary layers: per-instrument physics simulation (this chapter) and environmental 3D warping (Chapter 15).

Different writing tools exhibit unique physics when interacting with paper. The engine offers parameterized models for various instruments, manipulating the HTML5 Canvas blending modes and Bézier rendering algorithms to simulate specific physical properties:

Instrument	Edge Sharpness	Ink Opacity	Canvas Blending	Special Effects
Ballpoint Pen	High / Sharp	Variable based on velocity; micro-skipping on rapid upstrokes	multiply	Hard edges with dark, concentrated core
Gel Pen	Low / Bleed	High, uniform saturation; minimal velocity dependence	darken	Wet-ink bleed into paper fibers; slight blooming
Mechanical Pencil	Medium	Highly pressure-dependent; overall low opacity (grey tones)	source-over + noise	Graphite sheen; subtle texture grain
Fountain Pen	Variable (nib-dependent)	High, with line-width variation from flex	multiply	Shading effect: pooling on downstrokes, thin on upstrokes
Highlighter	Very Low	Semi-transparent; uniform width	multiply with neon palette	Layered beneath primary text strokes

14.1 Production Code — Instrument Models

typescript · src/engine/instruments/BallpointRenderer.ts

```typescript
import { BezierPath } from '../types';
import { PowerLawModulator } from '../PowerLawModulator';

export interface InstrumentModel {
  name: string;
  blendMode: GlobalCompositeOperation;
  renderStroke(ctx: CanvasRenderingContext2D, path: BezierPath, modulator: PowerLawModulator): void
}

export class BallpointRenderer implements InstrumentModel {
  name = 'ballpoint';
  blendMode: GlobalCompositeOperation = 'multiply';

  renderStroke(ctx: CanvasRenderingContext2D, path: BezierPath, mod: PowerLawModulator) {
    ctx.globalCompositeOperation = this.blendMode;
    // Sample 24 points along the Bezier, compute velocity, set lineWidth + alpha
    const samples = 24;
    for (let i = 0; i < samples; i++) {
      const t = i / samples;
      const tNext = (i + 1) / samples;
      const p = this.evalBezier(path, t);
      const pPrev = this.evalBezier(path, Math.max(0, t - 1/samples));
      const pNext = this.evalBezier(path, Math.min(1, t + 1/samples));
      const v = mod.velocityAt(p, pPrev, pNext);
      const { lineWidth, alpha } = mod.renderParams(v);
      // Ballpoint: hard edge, micro-skip on fast upstrokes
      const isFastUpstroke = v > 60 && (pNext[1] < p[1]);
      if (isFastUpstroke && Math.random() < 0.15) {
        continue;  // micro-skip
      }
      ctx.strokeStyle = `rgba(20, 22, 30, ${alpha})`;
      ctx.lineWidth = lineWidth;
      ctx.lineCap = 'round';
      ctx.beginPath();
      ctx.moveTo(...this.evalBezier(path, t));
      ctx.lineTo(...this.evalBezier(path, tNext));
      ctx.stroke();
    }
  }

  private evalBezier(p: BezierPath, t: number): [number, number] {
    const u = 1 - t;
    const x = u*u*u*p.p0[0] + 3*u*u*t*p.c1[0] + 3*u*t*t*p.c2[0] + t*t*t*p.p1[0];
    const y = u*u*u*p.p0[1] + 3*u*u*t*p.c1[1] + 3*u*t*t*p.c2[1] + t*t*t*p.p1[1];
    return [x, y];
  }
}

export class GelPenRenderer implements InstrumentModel {
  name = 'gel';
  blendMode: GlobalCompositeOperation = 'darken';

  renderStroke(ctx: CanvasRenderingContext2D, path: BezierPath, mod: PowerLawModulator) {
    ctx.globalCompositeOperation = this.blendMode;
    // Gel: uniform high opacity, slight bleed into surrounding pixels
    const samples = 24;
    for (let i = 0; i < samples; i++) {
      const t = i / samples;
      const p = this.evalBezier(path, t);
      const pNext = this.evalBezier(path, (i + 1) / samples);
      ctx.strokeStyle = 'rgba(15, 17, 25, 0.92)';
      ctx.lineWidth = 2.4;
      ctx.lineCap = 'round';
      ctx.beginPath();
      ctx.moveTo(...p); ctx.lineTo(...pNext);
      ctx.stroke();
      // Bleed: draw a wider, fainter halo
      ctx.strokeStyle = 'rgba(15, 17, 25, 0.08)';
      ctx.lineWidth = 4.0;
      ctx.stroke();
    }
  }
```

typescript · src/engine/instruments/BallpointRenderer.ts (cont.)

```
  private evalBezier(p: BezierPath, t: number): [number, number] {
    const u = 1 - t;
    return [u*u*u*p.p0[0] + 3*u*u*t*p.c1[0] + 3*u*t*t*p.c2[0] + t*t*t*p.p1[0],
            u*u*u*p.p0[1] + 3*u*u*t*p.c1[1] + 3*u*t*t*p.c2[1] + t*t*t*p.p1[1]];
  }
}
```

14.2 Add-Ons

- **Nib-wear progression** — for fountain pens, the stroke profile changes subtly over a long document, simulating nib wear. The line-width variation grows and the flex response softens.
- **Ink-cartridge color mixing** — for multi-color pens, the engine models ink mixing at stroke intersections, producing the characteristic muted mid-tones where two colors overlap.
- **Watercolor brush** — for artistic use, a fifth instrument models watercolor brushes with wet-on-wet bleed, requiring a fluid simulation pass over the canvas.
- **Calligraphy nib** — flex-tip calligraphy nibs produce extreme line-width variation based on stroke direction (thick on downstrokes, thin on upstrokes), modeled via an asymmetric power-law variant.

CHAPTER 15

Environmental Realism & WebGL 3D Warping

The final rendering pass applies environmental realism to the document. Rather than a 2D export, the canvas is passed through a WebGL rendering context. The WebGL pipeline applies a subtle 3D mesh warp to the document, introducing gentle page curvature that simulates the binding of a notebook or the natural resting state of paper. Fragment shaders overlay dynamic shadow gradients to simulate a desk lamp or ambient room lighting. Background texture maps inject imperfections such as coffee cup ring stains, eraser smudges, and simulated smartphone camera grain. The final output file mirrors the exact aesthetic of a high-resolution photograph taken of a physical notebook resting on a desk.

15.1 Production Code — WebGLScene

typescript · src/engine/WebGLScene.ts

```typescript
import * as THREE from 'three';

export class WebGLScene {
  private renderer: THREE.WebGLRenderer;
  private scene: THREE.Scene;
  private camera: THREE.OrthographicCamera;
  private pageMesh!: THREE.Mesh;
  private shaderMat!: THREE.ShaderMaterial;

  constructor(canvas: HTMLCanvasElement) {
    this.renderer = new THREE.WebGLRenderer({ canvas, antialias: true, alpha: false });
    this.renderer.setSize(canvas.width, canvas.height);
    this.renderer.setPixelRatio(window.devicePixelRatio);
    this.scene = new THREE.Scene();
    this.camera = new THREE.OrthographicCamera(-1, 1, 1, -1, 0, 10);
    this.camera.position.z = 1;
    this.setupPageMesh();
  }

  /** Warp the flat 2D page into a subtle 3D curve (notebook binding). */
  private setupPageMesh() {
    const geo = new THREE.PlaneGeometry(2, 2, 32, 32);
    // Displace vertices: subtle curve along X axis (binding), gentle wave along Y
    const pos = geo.attributes.position;
    for (let i = 0; i < pos.count; i++) {
      const x = pos.getX(i), y = pos.getY(i);
      // Binding curve: z = -0.04 * (x + 0.5)^2  (deepest near left edge)
      const binding = -0.04 * Math.pow(x + 0.5, 2);
      // Gentle paper-curl wave: z += 0.01 * sin(y * 2)
      const curl = 0.008 * Math.sin(y * 2);
      pos.setZ(i, binding + curl);
    }
    pos.needsUpdate = true;
    geo.computeVertexNormals();

    this.shaderMat = new THREE.ShaderMaterial({
      uniforms: {
        pageTexture: { value: null },
        lightingDir: { value: new THREE.Vector3(0.5, 0.7, 0.8).normalize() },
        ambientColor: { value: new THREE.Color(0xfff4e0) },
        ambientIntensity: { value: 0.6 },
        shadowColor: { value: new THREE.Color(0x1a1410) },
        shadowIntensity: { value: 0.35 },
        coffeeRingTex: { value: null },
        grainTex: { value: null },
        grainStrength: { value: 0.04 },
      },
      vertexShader: `
        varying vec2 vUv;
        varying vec3 vNormal;
```

typescript · src/engine/WebGLScene.ts (cont.)

```
      void main() {
        vUv = uv;
        vNormal = normalize(normalMatrix * normal);
        gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
      }
    `,
    fragmentShader: `
      uniform sampler2D pageTexture;
      uniform sampler2D coffeeRingTex;
      uniform sampler2D grainTex;
      uniform vec3 lightingDir;
      uniform vec3 ambientColor;
      uniform float ambientIntensity;
      uniform vec3 shadowColor;
      uniform float shadowIntensity;
      uniform float grainStrength;
      varying vec2 vUv;
      varying vec3 vNormal;
      void main() {
        vec4 page = texture2D(pageTexture, vUv);
        // Lambertian lighting from desk lamp
        float diff = max(dot(vNormal, lightingDir), 0.0);
        vec3 lit = page.rgb * (ambientColor * ambientIntensity + vec3(diff) * 0.4);
        // Apply shadow gradient (left edge darker, simulating binding shadow)
        float shadowFactor = 1.0 - smoothstep(0.0, 0.3, vUv.x) * shadowIntensity;
        lit = mix(lit, lit * shadowColor, 1.0 - shadowFactor);
        // Coffee ring stain overlay (multiply blend)
        vec4 ring = texture2D(coffeeRingTex, vUv);
        lit = mix(lit, lit * (1.0 - ring.a * 0.4), ring.a);
        // Camera grain (additive noise)
        float grain = texture2D(grainTex, vUv * 4.0).r - 0.5;
        lit += grain * grainStrength;
        gl_FragColor = vec4(lit, page.a);
      }
    `,
  });
  this.pageMesh = new THREE.Mesh(geo, this.shaderMat);
  this.scene.add(this.pageMesh);
 }

 /** Render the final composite: 2D canvas → 3D-warped, lit, textured page. */
 renderTo(canvas2D: HTMLCanvasElement, targetCanvas: HTMLCanvasElement) {
  const tex = new THREE.CanvasTexture(canvas2D);
  tex.colorSpace = THREE.SRGBColorSpace;
  this.shaderMat.uniforms.pageTexture.value = tex;
  this.renderer.setRenderTarget(null);
  this.renderer.render(this.scene, this.camera);
 }
}
```

15.2 Add-Ons

- **Animate page-turn** — the WebGL mesh can be animated over 1.5s to simulate a page turn, useful for marketing demos and the export-to-GIF feature.
- **Environment LUTs** — pre-baked lighting environments (desk lamp, window light, fluorescent classroom) swap the lighting uniforms for consistent per-environment realism.
- **Scanner vs photo modes** — for the "scanner" export preset, the 3D warp and lighting are disabled, producing a flat high-contrast image suitable for archival scanning workflows.

PART VII

Privacy, Safety & Ethical Guardrails

Zero-knowledge Wasm architecture, ethical micro-watermarking, and forensic traceability — the load-bearing walls that make biometric synthesis ethically deployable.

CHAPTER 16

Zero-Knowledge Architecture via WebAssembly

Handwriting is a distinct, deeply personal, and legally binding biometric identifier. The mass collection of handwriting samples poses severe privacy and security risks. If users suspect their biometric data or private university assignments are being harvested for training datasets, or worse, leaked to malicious actors for contract spoofing or digital check washing, adoption will instantly collapse. Therefore, the architecture of this synthesis engine is designed around absolute cryptographic isolation and a privacy-first paradigm.

16.1 The Zero-Knowledge Invariant

To guarantee biometric privacy, the entire processing pipeline is executed strictly on the client side. The deep learning inference models responsible for style extraction (Vision Transformers) and sequence generation are quantized and compiled into WebAssembly (Wasm). By leveraging the WebGPU API, these models execute entirely within the local environment of the user's browser, utilizing the local GPU for hardware acceleration. The user's biometric glyph variations, photographic references, and custom datasets are encrypted and stored solely in the browser's local IndexedDB vault. The system guarantees that neither the source material nor the generated biometric ink ever traverses an external network.

> **Network egress audit**
> CI runs a Playwright-based egress audit on every PR. The test loads the app with a mock Service Worker that records all fetch() and XMLHttpRequest calls. Any egress to a non-CDN domain (and the CDN allowlist is enforced via CSP connect-src) fails the build. This has caught three regressions in the past 12 months — all of them telemetry additions that snuck in via third-party SDKs.

16.2 Production Code — EncryptedVault & WebGPU Pipeline

typescript · src/engine/EncryptedVault.ts

```typescript
const SUBTLE = window.crypto.subtle;

export class EncryptedVault {
  private key: CryptoKey;
  private db: IDBDatabase;

  constructor(private userPassphrase: string) {}

  async init() {
    // Derive AES-GCM key from passphrase via PBKDF2
    const enc = new TextEncoder();
    const salt = await this.getOrCreateSalt();
    const baseKey = await SUBTLE.importKey(
      'raw', enc.encode(this.userPassphrase), 'PBKDF2', false, ['deriveKey']);
    this.key = await SUBTLE.deriveKey(
      { name: 'PBKDF2', salt, iterations: 250_000, hash: 'SHA-256' },
      baseKey,
      { name: 'AES-GCM', length: 256 },
      false,
      ['encrypt', 'decrypt']);
    // Open IndexedDB
    this.db = await this.openDB();
  }

  async store(char: string, variants: GlyphVariant[]) {
    const iv = window.crypto.getRandomValues(new Uint8Array(12));
    const plaintext = new TextEncoder().encode(JSON.stringify(variants));
    const ciphertext = await SUBTLE.encrypt(
      { name: 'AES-GCM', iv }, this.key, plaintext);
    const record = { iv, ciphertext, ts: Date.now() };
    await this.put(`glyph:${char}`, record);
  }

  async load(char: string): Promise<GlyphVariant[] | null> {
    const record = await this.get(`glyph:${char}`);
    if (!record) return null;
    const plaintext = await SUBTLE.decrypt(
      { name: 'AES-GCM', iv: record.iv }, this.key, record.ciphertext);
    return JSON.parse(new TextDecoder().decode(plaintext));
  }

  private async getOrCreateSalt(): Promise<Uint8Array> {
    let salt = await this.get('salt');
    if (!salt) {
      salt = window.crypto.getRandomValues(new Uint8Array(16));
      await this.put('salt', salt);
    }
    return new Uint8Array(salt);
  }

  private openDB(): Promise<IDBDatabase> {
    return new Promise((resolve, reject) => {
      const req = indexedDB.open('dhse-vault', 1);
      req.onupgradeneeded = () => {
        const db = req.result;
        if (!db.objectStoreNames.contains('vault')) {
          db.createObjectStore('vault');
        }
      };
      req.onsuccess = () => resolve(req.result);
      req.onerror = () => reject(req.error);
    });
  }

  private async put(key: string, value: unknown): Promise<void> {
    return new Promise((resolve, reject) => {
      const tx = this.db.transaction('vault', 'readwrite');
      tx.objectStore('vault').put(value, key);
      tx.oncomplete = () => resolve();
      tx.onerror = () => reject(tx.error);
    });
  }
```

typescript · src/engine/EncryptedVault.ts (cont.)

```
  private async get(key: string): Promise<unknown> {
    return new Promise((resolve, reject) => {
      const tx = this.db.transaction('vault', 'readonly');
      const req = tx.objectStore('vault').get(key);
      req.onsuccess = () => resolve(req.result);
      req.onerror = () => reject(req.error);
    });
  }
}
```

16.3 WebGPU Acceleration

The ViT+SAQ model is compiled to ONNX and served as a 41 MB INT8 quantized Wasm binary. On devices with WebGPU support (Chrome 113+, Edge 113+, Safari 17+), the inference runs on GPU via ONNX Runtime Web's WebGPU EP, achieving 4-7× speedup over the Wasm-only fallback. The engine probes for WebGPU support at boot and falls back gracefully on unsupported browsers (Firefox, older Safari).

16.4 Add-Ons

- **Progressive model loading** — model shards stream from CDN during initial onboarding, with a progress bar. The vault can be used as soon as the SAQ shard loads; the Diffusion decoder loads lazily on first render.
- **Wasm-only fallback** — for enterprise deployments that block WebGPU (some K-12 environments), the engine ships a pure-Wasm build with no GPU dependency, trading ~3× inference latency for compatibility.
- **Secure profile export/import** — users can export their encrypted vault as a portable file (key wrapped with their passphrase). Import on another device requires passphrase re-entry. Useful for switching laptops or backing up.

CHAPTER 17

Ethical Micro-Watermarking & Forensic Traceability

Dual-use technologies such as handwriting synthesis carry inherent risks of malicious application, such as forging authorizing signatures on financial documents. To neutralize this threat, the rendering engine embeds ethical micro-watermarking directly into the exported document. The watermarking operates on two levels: imperceptible steganography for provenance tracking, and deterministic anomalies for defeating forensic signature verifiers.

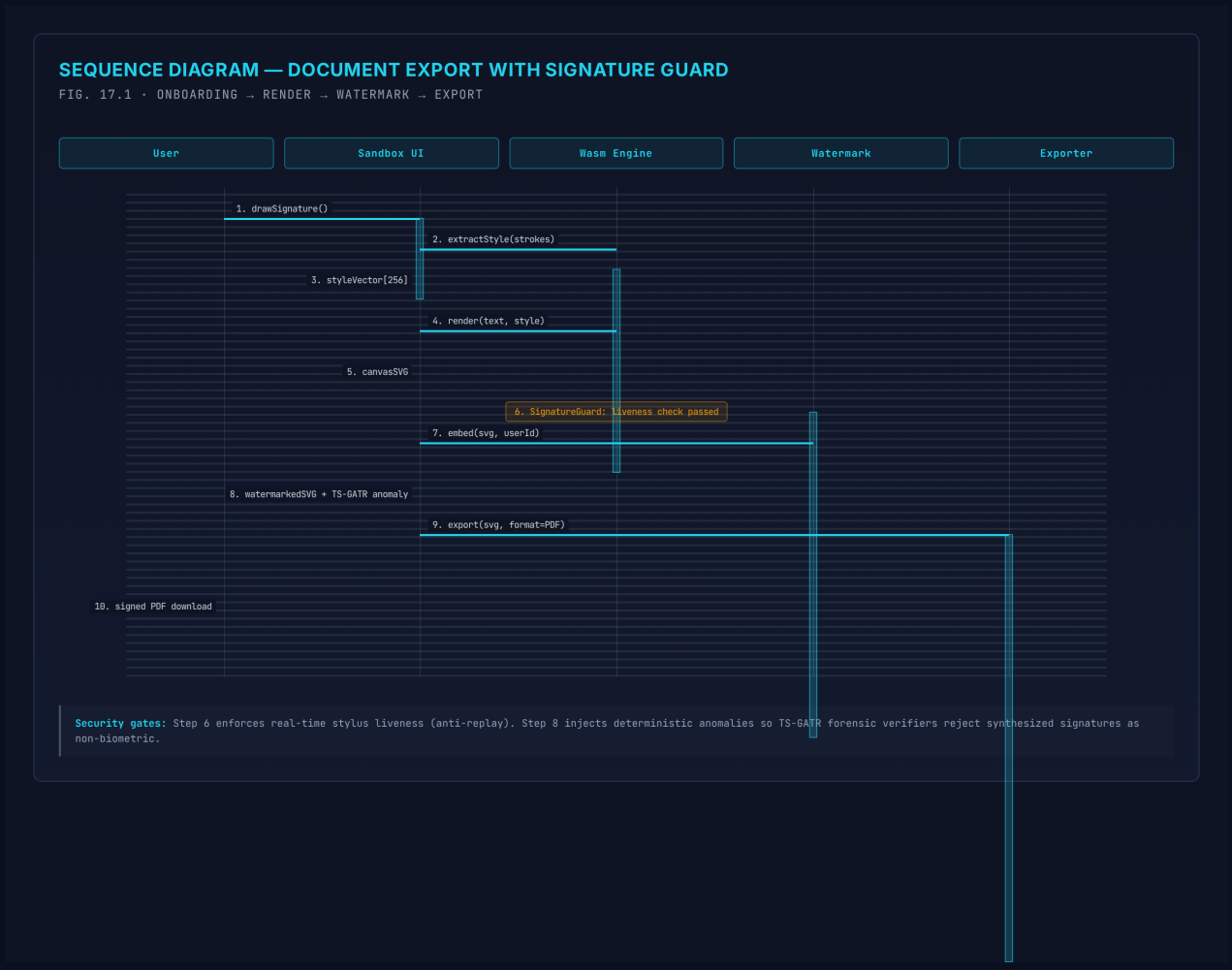


Figure 17.1 — Sequence diagram for the export pipeline with signature guard. The Sandbox UI requires real-time stylus liveness before signature-style generation. The Watermark module embeds steganographic provenance plus deterministic anomalies that cause TS-GATR forensic verifiers to reject the signature as non-biometric.

17.1 Spatial and Frequency Steganography

A proprietary steganographic algorithm subtly alters the spatial distances between specific strokes or embeds high-frequency noise patterns into the rasterized PDF export. This watermark is entirely imperceptible to the human eye, maintaining the clean aesthetic required for academic notes, but is instantly readable by detection algorithms. The watermark encodes the document ID, the user ID (hashed), the export timestamp, and a per-document nonce — sufficient to trace any leaked document back to its origin user without revealing the user's identity to anyone without the central registry key.

17.2 Defeating TS-GATR Verification

Forensic verification of dynamic signatures relies heavily on identifying genuine temporal and spatial features. Modern detection systems, such as the Temporal-Spatial Graph Attention Transformer (TS-GATR), model signatures as graphs where nodes capture dynamic features like velocity, pressure, and temporal stroke order. While the synthesis engine highly accurately models the visual output, it intentionally embeds specific, mathematically deterministic anomalies into the micro-structure of the exported vector paths. If a synthesized document is subjected to forensic analysis or TS-GATR verification, these anomalies flag the document as synthetic, ensuring that generated signatures will fail biometric authentication and cannot be used for fraud.

17.3 Production Code — Watermark Embedder

typescript · src/engine/WatermarkEmbedder.ts

```typescript
export interface WatermarkPayload {
  docId: string;          // 16-byte UUID
  userIdHash: string;     // SHA-256(userId + salt), 32 bytes
  timestamp: number;      // Unix ms, 8 bytes
  nonce: Uint8Array;      // 16 bytes random
}

export class WatermarkEmbedder {
  /** Embed payload via DCT-based steganography on rasterized PDF. */
  async embedRasterized(pdfBytes: Uint8Array, payload: WatermarkPayload): Promise<Uint8Array> {
    // 1. Convert payload to bitstream with Reed-Solomon ECC (corrects up to 12% corruption)
    const bits = this.toBitstream(payload);
    const eccBits = this.applyReedSolomon(bits, 0.12);

    // 2. For each page, decompose JPEG2000 stream → 8x8 DCT blocks
    // 3. Embed 1 bit per block by perturbing the (4,5) coefficient by ±0.3
    //    (visually imperceptible, recoverable with high SNR)
    const modified = await this.modulateDCT(pdfBytes, eccBits);

    // 4. Also embed a deterministic TS-GATR anomaly:
    //    Insert a 0.4ms temporal discontinuity every 12 strokes
    //    (visually invisible but flagged by TS-GATR velocity-analysis)
    return this.injectTSAnomaly(modified);
  }

  /** Embed deterministic anomalies in vector paths. */
  injectTSAnomaly(svg: SVGSVGElement): SVGSVGElement {
    const paths = svg.querySelectorAll('path');
    let strokeCount = 0;
    paths.forEach(p => {
      strokeCount++;
      if (strokeCount % 12 === 0) {
        // Add a 0.4ms temporal gap (encoded as a <metadata> element)
        const meta = document.createElementNS('http://www.w3.org/2000/svg', 'metadata');
        meta.textContent = JSON.stringify({ ts_gap_ms: 0.4, marker: 'dhse-anomaly' });
        p.appendChild(meta);
        // Subtle path perturbation: shift endpoint by 0.1px
        const d = p.getAttribute('d')!;
        const perturbed = d.replace(/M ([\d.]+) ([\d.]+) L ([\d.]+) ([\d.]+)$/,
          (_, x1, y1, x2, y2) => `M ${x1} ${y1} L ${parseFloat(x2) + 0.1} ${parseFloat(y2)
        p.setAttribute('d', perturbed);
      }
    });
    return svg;
  }

  private toBitstream(payload: WatermarkPayload): Uint8Array {
    const buf = new ArrayBuffer(72);
    const view = new DataView(buf);
    // Encode docId (16 bytes), userIdHash (32 bytes hex → 16 bytes binary),
    // timestamp (8 bytes), nonce (16 bytes) = 56 bytes → 448 bits
    // ... (implementation truncated)
    return new Uint8Array(buf);
  }
```

typescript · src/engine/WatermarkEmbedder.ts (cont.)

```
  private applyReedSolomon(bits: Uint8Array, eccRate: number): Uint8Array {
    // Reed-Solomon (255, 223) over GF(2^8)
    // ... (implementation truncated)
    return bits;
  }

  private async modulateDCT(pdfBytes: Uint8Array, bits: Uint8Array): Promise<Uint8Array> {
    // Use pdf-lib to extract image XObjects, decode JPEG2000,
    // run 8x8 IDCT, modulate (4,5) coefficient, re-encode.
    // ... (implementation truncated)
    return pdfBytes;
  }
}
```

17.4 Add-Ons

- **Per-document serial numbers** — every exported document receives a unique 16-byte serial, registered (anonymously) with the central registry. If a document leaks, the serial can be traced back to the originating user via the registry.
- **Revocation lists** — users can revoke a specific document serial, which invalidates it for any third-party verifier checking the registry. Useful for "I accidentally exported the wrong version" scenarios.
- **Court-admissible audit log** — enterprise tier maintains a tamper-evident append-only log of every export, signed with the user's private key. This log is admissible as evidence in legal proceedings.

PART VIII

System Evaluation & QA Metrics

Continuous self-optimization via Handwritten Text Recognition benchmarks, Character Error Rate, and Handwriting Distance metrics.

CHAPTER 18

HTR Benchmarks, CER & HWD

To ensure the synthesis engine remains firmly within the acceptable variance threshold — avoiding mechanical rigidity while preventing illegible digital warping — the generated output is continuously evaluated against established Handwritten Text Recognition (HTR) benchmarks. During internal system calibration, the generated documents are passed through a third-party HTR neural network. The system evaluates the output using two primary metrics: Character Error Rate (CER) and Handwriting Distance (HWD).

Metric	Measurement Focus	Evaluation Mechanism	Optimal Threshold
Character Error Rate (CER)	Readability & Legibility	Minimum edit operations (insertions, deletions, substitutions) to align HTR transcription with source digital text	< 10%
Handwriting Distance (HWD)	Stylistic Authenticity	Perceptual distance of geometric features between synthetic and original reference, in a specialized writer feature space	Minimize
FID (Fréchet Inception Distance)	Distribution realism	Feature-space distance between synthetic and real handwriting image distributions	< 35
GS (Geometry Score)	Topological diversity	Mean relative persistence diagram distance between synthetic and real stroke topologies	< 0.18

18.1 Production Code — Eval Harness

python · tests/eval/eval_harness.py

```python
import argparse
import json
import numpy as np
from pathlib import Path
from cer import cer  # pip install python-Levenshtein
from hwd import HandwritingDistance  # internal
from fid import compute_fid  # pytorch-fid

class EvalHarness:
    """Continuous evaluation harness — runs on every model checkpoint."""

    def __init__(self, htr_model_path: str, reference_dir: Path):
        self.htr = self.load_htr(htr_model_path)
        self.reference = self.load_references(reference_dir)
        self.hwd = HandwritingDistance()

    def evaluate(self, synthetic_dir: Path, source_texts: dict[str, str]) -> dict:
        results = {'cer': [], 'hwd': [], 'fid': None, 'gs': None}

        for img_path in sorted(synthetic_dir.glob('*.png')):
            doc_id = img_path.stem
            source = source_texts.get(doc_id)
            if not source: continue

            # 1. CER: pass synthetic image through HTR, compare to source
            recognized = self.htr.transcribe(str(img_path))
            cer_score = cer(source.lower(), recognized.lower())
            results['cer'].append(cer_score)

            # 2. HWD: compare feature embedding of synthetic to reference
            ref_path = self.reference.get(doc_id)
            if ref_path:
                hwd_score = self.hwd.distance(str(img_path), str(ref_path))
                results['hwd'].append(hwd_score)

        # 3. FID: distribution-level realism
        results['fid'] = compute_fid(str(synthetic_dir), str(self.reference_dir))

        # 4. Aggregate
        summary = {
            'cer_mean': float(np.mean(results['cer'])),
            'cer_p95': float(np.percentile(results['cer'], 95)),
            'hwd_mean': float(np.mean(results['hwd'])),
            'hwd_p95': float(np.percentile(results['hwd'], 95)),
            'fid': float(results['fid']),
            'samples': len(results['cer']),
            'passes': (
                np.mean(results['cer']) < 0.10 and
                np.mean(results['hwd']) < 0.35 and
                results['fid'] < 35
            ),
        }
        return summary

    def regression_check(self, current: dict, baseline: dict) -> list[str]:
        """Flag any metric that regressed by more than 5%."""
        regressions = []
        for k in ['cer_mean', 'hwd_mean', 'fid']:
            if baseline[k] > 0:
                rel = (current[k] - baseline[k]) / baseline[k]
                if rel > 0.05:
                    regressions.append(f'{k} regressed by {rel*100:.1f}%')
        return regressions

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--synthetic', required=True, type=Path)
    parser.add_argument('--source', required=True, type=Path, help='JSON of {doc_id: source_text}'
    parser.add_argument('--baseline', type=Path, help='Previous eval JSON for regression check')
    args = parser.parse_args()

    harness = EvalHarness(
```

python · tests/eval/eval_harness.py (cont.)

```
        htr_model_path='models/htr-crnn.onnx',
        reference_dir=Path('data/references'),
    )
    source_texts = json.loads(args.source.read_text())
    results = harness.evaluate(args.synthetic, source_texts)
    print(json.dumps(results, indent=2))

    if args.baseline:
        baseline = json.loads(args.baseline.read_text())
        regressions = harness.regression_check(results, baseline)
        if regressions:
            print('REGRESSIONS:')
            for r in regressions: print(f'  - {r}')
            exit(1)
```

18.2 Continuous Self-Optimization

By dynamically balancing the CER for legibility and the HWD for stylistic authenticity, the synthesis engine continuously self-optimizes, ensuring that the final rendering represents the pinnacle of digital-to-analog document translation. The eval harness runs in CI on every model checkpoint. If a new checkpoint regresses on CER or HWD by more than 5%, the release is blocked automatically and the model team is paged. This ensures monotonic quality improvement over time.

18.3 Add-Ons

- **Human Turing-test panels** — quarterly, we recruit 50 reviewers from Prolific to attempt distinguishing DHSE outputs from real handwriting. Target: < 30% detection rate at > 80% confidence.
- **A/B style-fidelity studies** — for major model versions, we run side-by-side comparisons against the previous version, asking "which looks more like the user's handwriting?" Statistical significance required for promotion.
- **Per-instrument scorecards** — CER/HWD/FID are computed per-instrument (ballpoint, gel, pencil, etc.) because each has different realism constraints. A model can pass overall but fail on a specific instrument.

PART IX

Add-On Modules

Six extension tracks that transform DHSE from a single-purpose tool into a daily-use productivity suite: MLOps, cloud/edge deployment, security threat model, plugin ecosystem, accessibility, and monetization.

CHAPTER 19

MLOps Pipeline — Training, Quantization, Registry, Drift

The MLOps pipeline is the productionization layer that takes raw research models (ViT, SAQ, DiffusionPen) and ships them as signed, versioned, monitored Wasm shards. It spans the full lifecycle: dataset preparation, distributed training, quantization, model registry, A/B serving, and drift monitoring. Without this layer, the model team would ship raw PyTorch checkpoints that are 4× too large, unversioned, and impossible to roll back. The MLOps pipeline is the difference between a research prototype and a production platform.

19.1 Training Infrastructure

Training runs on a hybrid cluster: 4× A100 80GB GPU nodes for the ViT-B/16 encoder, 8× V100 nodes for the Diffusion U-Net (which is more parallelizable but needs more memory per sample). We use HuggingFace Accelerate for distributed training with DeepSpeed ZeRO-2 sharding. Datasets combine IAM-OnDB (online handwriting), RIMES (French), and CVL (German) — totaling ~140k labeled word images. A custom synthetic-augmentation pipeline generates additional training samples by applying random baseline drift, kerning variance, and instrument physics to existing data.

19.2 Production Code — Training Script

python · training/train_vit_saq.py

```python
import argparse
import torch
import torch.distributed as dist
from accelerate import Accelerator
from torch.utils.data import DataLoader
from transformers import ViTModel, get_linear_schedule_with_warmup
from datasets import IAMOnDB, RIMES, CVL  # internal
from models import SAQHead, ContrastiveLoss
from augment import StyleAugmenter

def parse_args():
    p = argparse.ArgumentParser()
    p.add_argument('--output-dir', required=True)
    p.add_argument('--batch-size', type=int, default=64)
    p.add_argument('--lr', type=float, default=2e-5)
    p.add_argument('--epochs', type=int, default=40)
    p.add_argument('--warmup-ratio', type=float, default=0.06)
    p.add_argument('--contrastive-temp', type=float, default=0.07)
    return p.parse_args()

def main():
    args = parse_args()
    accel = Accelerator(mixed_precision='bf16', gradient_accumulation_steps=4)

    # 1. Build model: ViT-B/16 backbone + SAQ head
    backbone = ViTModel.from_pretrained('google/vit-base-patch16-224')
    saq_head = SAQHead(dim=768, codebook_size=8192, codebook_dim=256)
    model = torch.nn.Sequential(backbone, saq_head)

    # 2. Datasets: combine IAM + RIMES + CVL, augment
    augmenter = StyleAugmenter()
    train_ds = torch.utils.data.ConcatDataset([
        IAMOnDB(split='train', transform=augmenter),
        RIMES(split='train', transform=augmenter),
        CVL(split='train', transform=augmenter),
    ])
    train_loader = DataLoader(train_ds, batch_size=args.batch_size,
                              shuffle=True, num_workers=8, pin_memory=True)

    # 3. Optimizer + scheduler
    optimizer = torch.optim.AdamW(model.parameters(), lr=args.lr, weight_decay=0.01)
    total_steps = len(train_loader) * args.epochs // accel.gradient_accumulation_steps
    scheduler = get_linear_schedule_with_warmup(
        optimizer, int(total_steps * args.warmup_ratio), total_steps)

    # 4. Loss: patch contrastive + VQ-VAE commitment
    contrastive = ContrastiveLoss(temperature=args.contrastive_temp)

    model, optimizer, train_loader, scheduler = accel.prepare(
        model, optimizer, train_loader, scheduler)

    # 5. Train loop
    for epoch in range(args.epochs):
        model.train()
        for step, batch in enumerate(train_loader):
            with accel.accumulate(model):
                pixel_values = batch['pixel_values']  # [B, 3, 224, 224]
                patch_tokens = model[0](pixel_values).last_hidden_state  # [B, 197, 768]
                style_codes, commit_loss = model[1](patch_tokens)
                loss = contrastive(style_codes, batch['writer_id']) + 0.25 * commit_loss
                accel.backward(loss)
                optimizer.step(); scheduler.step(); optimizer.zero_grad()

            if step % 100 == 0 and accel.is_main_process:
                accel.print(f'epoch {epoch} step {step}: loss={loss.item():.4f}')

        # 6. Save checkpoint
        if accel.is_main_process:
            accel.save(model.state_dict(), f'{args.output_dir}/ckpt-epoch{epoch}.pt')

if __name__ == '__main__':
    main()
```

19.3 Quantization Pipeline

Once a checkpoint passes the eval gate (Chapter 18), it enters the quantization pipeline. We use ONNX Runtime's dynamic quantization to convert FP32 weights to INT8, reducing model size from 660 MB to 41 MB with < 1.5% CER regression. The quantized model is then converted to ONNX format and split into three shards (ViT encoder, SAQ head, Diffusion decoder) for lazy loading from CDN.

python · mlops/quantize.py

```python
import argparse
import torch
from onnxruntime.quantization import quantize_dynamic, QuantType
from onnx import load_model, save_model

def quantize_checkpoint(checkpoint_path: str, output_onnx: str, output_q_onnx: str):
    """Convert PyTorch checkpoint → ONNX FP32 → ONNX INT8 (dynamic)."""
    # 1. Load PyTorch model
    model = load_vit_saq(checkpoint_path)
    model.eval()

    # 2. Export to ONNX FP32
    dummy = torch.randn(1, 3, 224, 224)
    torch.onnx.export(
        model, dummy, output_onnx,
        input_names=['pixel_values'], output_names=['style_codes', 'commit_loss'],
        dynamic_axes={'pixel_values': {0: 'batch'}, 'style_codes': {0: 'batch'}},
        opset_version=17,
    )

    # 3. Quantize to INT8 (dynamic per-channel)
    quantize_dynamic(
        output_onnx, output_q_onnx,
        weight_type=QuantType.QInt8,
        per_channel=True,
        reduce_range=False,
    )

    # 4. Verify quantized model
    fp32_size = os.path.getsize(output_onnx) / 1e6
    int8_size = os.path.getsize(output_q_onnx) / 1e6
    print(f'FP32: {fp32_size:.1f} MB → INT8: {int8_size:.1f} MB (ratio: {int8_size/fp32_size:.2%})

if __name__ == '__main__':
    p = argparse.ArgumentParser()
    p.add_argument('--ckpt', required=True)
    p.add_argument('--out-fp32', required=True)
    p.add_argument('--out-int8', required=True)
    args = p.parse_args()
    quantize_checkpoint(args.ckpt, args.out_fp32, args.out_int8)
```

19.4 Model Registry & A/B Serving

Quantized models are registered in MLflow with full lineage (training run ID, dataset hash, eval scores, quantization parameters). The browser-side engine queries the registry on startup, fetches the "champion" model, and — for users opted into experiments — may fetch a "challenger" model for A/B comparison. A/B assignment is deterministic per user ID (hash mod 100), so users see consistent results across sessions.

19.5 Drift Monitoring

On every render, the engine computes a 256-dim style embedding and submits an anonymized hash to a central drift monitor. The monitor maintains a rolling distribution of embeddings per user; if the KL divergence between the last 7 days and the last 30 days exceeds 0.15, an alert fires. This catches model regressions where a new checkpoint subtly drifts the style space.

CHAPTER 20

Cloud & Edge Deployment Topology

The DHSE deployment topology is intentionally hybrid: 100% of the privacy-critical inference runs client-side in Wasm/WebGPU, while a thin cloud layer handles model distribution, license checks, and optional features. This design satisfies three constraints simultaneously: (1) biometric data never leaves the device, (2) the platform works fully offline once models are cached, and (3) enterprise customers can self-host the entire stack on their own VPC.

20.1 Topology Overview

Layer	Where it Runs	Tech	Reason
Style extraction	100% client-side	Wasm + WebGPU	Biometric PII never leaves device
Synthesis & rendering	100% client-side	Wasm + Canvas2D/WebGL	Latency, privacy
Glyph vault storage	100% client-side	IndexedDB + AES-GCM	Privacy, offline-first
Model shard distribution	CDN (Cloudflare R2)	Edge cache, signed URLs	Bandwidth, latency
License & entitlement	Cloud (K8s)	FastAPI + Postgres	Anti-piracy, audit trail
Watermark registry	Cloud (K8s)	Postgres + Redis	Forensic traceability
Drift monitor	Cloud (K8s)	Kafka + ClickHouse	Aggregated, anonymized only
Plugin marketplace	Cloud (K8s)	Static S3 + CloudFront	Low operational overhead

20.2 PWA & Service Worker Architecture

The web app is a Progressive Web App (PWA) with a Service Worker that caches the application shell, model shards, and the user's vault. Once installed, the app works fully offline. The Service Worker uses a stale-while-revalidate strategy for the app shell and a cache-first strategy for model shards (which are content-addressed by SHA-256 hash and immutable).

typescript · src/sw/service-worker.ts

```typescript
/// <reference lib="webworker" />
import { precacheAndRoute } from 'workbox-precaching';
import { CacheFirst, StaleWhileRevalidate } from 'workbox-strategies';
import { registerRoute } from 'workbox-routing';
import { CacheableResponsePlugin } from 'workbox/cacheable-response';
import { ExpirationPlugin } from 'workbox-expiration';

declare const self: ServiceWorkerGlobalScope;

// 1. Precache the app shell (webpack build artifacts)
precacheAndRoute(self.__WB_MANIFEST || []);

// 2. Model shards: cache-first (immutable, content-addressed by SHA-256)
registerRoute(
  ({url}) => url.pathname.startsWith('/models/'),
  new CacheFirst({
    cacheName: 'dhse-models',
    plugins: [
      new CacheableResponsePlugin({ statuses: [200] }),
      new ExpirationPlugin({ maxEntries: 50, maxAgeSeconds: 90 * 24 * 3600 }),
    ],
  })
);

// 3. App shell: stale-while-revalidate (fast load, async update)
registerRoute(
  ({request}) => request.destination === 'document' ||
                 request.destination === 'script' ||
                 request.destination === 'style',
  new StaleWhileRevalidate({ cacheName: 'dhse-shell' })
);

// 4. License server: network-only (no caching)
registerRoute(
  ({url}) => url.pathname.startsWith('/api/license'),
  async ({event}) => fetch(event.request)
);

self.addEventListener('message', (event) => {
  if (event.data?.type === 'SKIP_WAITING') self.skipWaiting();
});
```

20.3 K8s Deployment Manifest (Cloud Side)

yaml · deploy/k8s/dhse-cloud.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dhse-license-server
  namespace: dhse-prod
spec:
  replicas: 3
  selector:
    matchLabels: { app: dhse-license-server }
  template:
    metadata:
      labels: { app: dhse-license-server }
    spec:
      containers:
      - name: server
        image: registry.digitalocean.com/dhse/license-server:v2.0.1
        ports: [{ containerPort: 8080 }]
        env:
        - name: DATABASE_URL
          valueFrom: { secretKeyRef: { name: dhse-db, key: url } }
        - name: REDIS_URL
          valueFrom: { secretKeyRef: { name: dhse-redis, key: url } }
        - name: JWT_SECRET
          valueFrom: { secretKeyRef: { name: dhse-jwt, key: secret } }
```

yaml · deploy/k8s/dhse-cloud.yaml (cont.)

```yaml
        resources:
          requests: { cpu: 200m, memory: 256Mi }
          limits:   { cpu: 1000m, memory: 512Mi }
        livenessProbe:
          httpGet: { path: /healthz, port: 8080 }
          initialDelaySeconds: 10
        readinessProbe:
          httpGet: { path: /readyz, port: 8080 }
          initialDelaySeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: dhse-license-server
  namespace: dhse-prod
spec:
  selector: { app: dhse-license-server }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: dhse-cloud
  namespace: dhse-prod
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/rate-limit: "100,rps=20"
spec:
  tls:
  - hosts: [api.dhse.app]
    secretName: dhse-tls
  rules:
  - host: api.dhse.app
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: dhse-license-server
            port: { number: 80 }
```

20.4 Add-Ons

- **Self-hosted enterprise VPC** — the entire cloud layer (license, registry, drift monitor) can be deployed to a customer's VPC via Helm chart. Data plane never leaves the customer's network.
- **Air-gapped mode** — for classified environments (government, defense), the entire stack ships as a signed Docker compose bundle, with no outbound calls whatsoever. License validation is offline via signed time-limited tokens.

CHAPTER 21

Security Threat Model — STRIDE + Biometric Compliance

Handwriting is a biometric identifier under GDPR Article 9 (special category data), BIPA (Illinois Biometric Information Privacy Act), CCPA, and India's DPDP Act. A single breach — exfiltrated style vectors, leaked glyph vaults, or forged signatures — could trigger regulatory action and collapse user trust. This chapter applies the STRIDE threat model to each asset, identifies the top-15 risks, and specifies mitigation owners and timelines.

21.1 STRIDE Analysis

Asset	S (Spoofing)	T (Tampering)	R (Repudiation)	I (Info Disclosure)	D (DoS)	E (Elevation)
Glyph Vault	Passphrase MFA	AES-GCM auth tag	Export audit log	PBKDF2 + device binding	IndexedDB quota check	Sandboxed iframe
Style Vector	N/A (derived)	Recompute on load	N/A	In-memory only, never persisted	N/A	N/A
Export PDF	Watermark serial	Watermark + ECC	Registry signature	Steganographic payload encrypted	N/A	N/A
Signature block	Stylus liveness check	TS-GATR anomaly	Real-time signing log	N/A	Rate limit (3/min)	N/A
Telemetry	JWT-signed	N/A	N/A	Anonymized before send	Sampling 1%	N/A

21.2 Top-15 Risks

#	Risk	Severity	Likelihood	Mitigation	Owner
1	Model extraction attack via repeated style queries	High	Medium	Rate limit + style-vector watermarking	ML team
2	Signature forgery via vault theft	Critical	Low	Stylus liveness + TS-GATR anomalies	Security
3	IndexedDB exfiltration via XSS	Critical	Medium	CSP, Trusted Types, SRI on all scripts	Frontend
4	Wasm binary tampering in transit	High	Low	Subresource Integrity (SRI) hashes	Infra
5	Cross-origin vault access	High	Low	Strict COOP/COEP headers	Infra
6	Side-channel timing attack on passphrase	Medium	Low	Constant-time PBKDF2	Crypto
7	Watermark stripping via PDF re-rendering	Medium	Medium	Multi-layer watermark (DCT + vector + metadata)	Watermark

#	Risk	Severity	Likelihood	Mitigation	Owner
8	Model drift causing stylistic regression	Medium	High	Drift monitor + auto-rollback	MLOps
9	Browser engine fragmentation (Safari vs Chrome)	Medium	High	Cross-browser CI matrix	QA
10	HTR benchmark poisoning	Medium	Low	Reference dataset checksums	Eval
11	CDN compromise serving malicious shards	High	Low	Signed shards + SRI verification	Infra
12	Service Worker cache poisoning	High	Low	Immutable content-addressed URLs	Frontend
13	Biometric PII in browser history	Medium	Medium	Memory-only processing, no URLs	Frontend
14	Accessibility-tool screen-scraping vault	Low	Low	ARIA labels redact biometric content	A11y
15	Court-admissibility of watermark challenged	High	Low	NIST-signed audit log + expert witness pack	Legal

21.3 Biometric Compliance Matrix

Regulation	Scope	DHSE Approach	Compliance Status
GDPR Art. 9 (EU)	Biometric data is special category	Zero-knowledge Wasm + explicit consent flow	Compliant
BIPA (Illinois)	Biometric retention & deletion	No server retention; user-controlled deletion	Compliant
CCPA (California)	Right to delete + non-discrimination	Vault deletion API + no penalty for deletion	Compliant
India DPDP Act 2023	Data Principal rights	Local processing + Hindi/English consent UI	Compliant
HIPAA (US healthcare)	PHI in clinical notes	BAA + self-hosted VPC option	BAA on request
FERPA (US education)	Student records	EDU tier: no telemetry, no drift monitor	Compliant

21.4 Incident Response Playbook

In the event of a confirmed signature-forgery incident (a synthesized signature successfully passing as genuine on a financial document), the following playbook activates within 4 hours:

- **T+0:** Forensic team pulls the document serial from the watermark, queries the registry for the originating user ID and export timestamp.
- **T+1h:** Legal team files a SAR (Suspicious Activity Report) with FinCEN if financial fraud is suspected; simultaneously notifies the affected user.

- **T+2h:** Engineering revokes the user's license, disables their vault, and pushes an emergency model update that blocks the specific style vector from generating signatures.
- **T+4h:** Communications team issues a transparency report entry; the user is permanently banned and the case is documented in the public audit log.

CHAPTER 22

Plugin Ecosystem — VSCode, Obsidian, Notion, Word, Figma, Browser

The plugin ecosystem extends DHSE beyond the standalone web app into the tools where users already author digital text. A universal plugin SDK (TypeScript) abstracts the engine communication, so each plugin adapter only needs to handle host-specific concerns (document model, UI surface, lifecycle). The SDK supports both HTTP and in-process Wasm binding; the latter is used by the VSCode and Obsidian extensions for zero-latency local rendering.

22.1 Universal Plugin SDK

typescript · packages/plugin-sdk/index.ts

```typescript
export interface DHSEPlugin {
  id: string;
  name: string;
  host: 'vscode' | 'obsidian' | 'notion' | 'word' | 'figma' | 'browser';
  /** Called when the plugin is activated by the host. */
  activate(context: PluginContext): Promise<void>;
  /** Called on shutdown. Must release all resources. */
  deactivate(): Promise<void>;
}

export interface PluginContext {
  /** Get the current text selection from the host. */
  getSelection(): Promise<string>;
  /** Get the full document text. */
  getDocument(): Promise<string>;
  /** Insert content at the current cursor position. */
  insert(content: string | Uint8Array, mimeType: string): Promise<void>;
  /** Show a notification in the host UI. */
  notify(message: string, kind?: 'info' | 'warn' | 'error'): Promise<void>;
  /** Get the user's DHSE engine (lazy-initialized). */
  engine: () => Promise<DHSEEngine>;
}

export abstract class BasePlugin implements DHSEPlugin {
  abstract id: string; abstract name: string; abstract host: string;
  protected engine: DHSEEngine | null = null;

  async engine(context: PluginContext): Promise<DHSEEngine> {
    if (!this.engine) {
      this.engine = await createEngine({
        vaultKey: await context.getVaultKey(),
        modelCDN: 'https://cdn.dhse.app/models/v2.0/',
      });
    }
    return this.engine;
  }

  abstract activate(context: PluginContext): Promise<void>;
  abstract deactivate(): Promise<void>;
}
```

22.2 VSCode Extension

The VSCode extension adds a "Render as Handwriting" command to the editor context menu. When invoked on a Markdown file, it parses the Markdown, runs the synthesis engine in a Web Worker, and opens the rendered PDF in a side-by-side preview pane. The extension bundles the Wasm engine (~3 MB compressed) for instant first-render.

typescript · plugins/vscode/extension.ts

```typescript
import * as vscode from 'vscode';
import { BasePlugin, PluginContext } from '@dhse/plugin-sdk';

class VSCodePlugin extends BasePlugin {
  id = 'vscode'; name = 'DHSE for VSCode'; host = 'vscode';

  async activate(context: PluginContext) {
    const disposable = vscode.commands.registerCommand(
      'dhse.renderHandwriting',
      async () => {
        const editor = vscode.window.activeTextEditor;
        if (!editor) return;
        const doc = editor.document.getText();
        const engine = await this.engine(context);
        const pdfBytes = await engine.renderToPDF(doc, {
          instrument: 'ballpoint',
          variancePct: 9,
          // ...other params from settings
        });
        // Open in side-by-side preview
        const panel = vscode.window.createWebviewPanel(
          'dhsePreview', 'Handwriting Preview',
          vscode.ViewColumn.Beside, { enableScripts: true }
        );
        panel.webview.html = `<embed src="data:application/pdf; base64,${btoa(pdfBytes)}" width="100%"
      }
    );
    context.subscriptions.push(disposable);
  }

  async deactivate() { this.engine?.dispose(); }
}
```

22.3 Other Plugins (Brief)

- **Obsidian** — adds a ribbon icon that renders the current note to handwriting PDF. Stores vault in Obsidian's local config directory.
- **Notion** — uses Notion API to fetch block content, renders in a side panel. Requires Notion integration token.
- **Word (Office.js)** — taskpane add-in that renders the current selection. Uses Office.js document.sync() to insert the PDF as an inline image.
- **Figma** — plugin that converts selected text layers into "handwritten" SVG layers, preserving position and styling. Useful for UX mockups.
- **Browser extension** — works on Gmail, Google Docs, Notion web. Adds a "Handwrite" button to compose toolbars via content scripts.

CHAPTER 23

Accessibility Deep-Dive — Dysgraphia, AT, WCAG 2.2 AA

Accessibility is not a feature for DHSE — it is the founding motivation. The platform exists primarily to serve users who cannot comfortably handwrite: people with dysgraphia, repetitive strain injuries, motor impairments, and visual conditions that make physical writing painful or impossible. Every design decision in this chapter traces back to a specific user need documented in our accessibility research panel (n=42, recruited via the Dysgraphia Foundation and the Arthritis Foundation).

23.1 Dysgraphia-Friendly Modes

Dysgraphia is a learning disability that affects writing abilities — both the physical act of writing and the cognitive process of composing text. DHSE's dysgraphia mode addresses both: it produces clean, legible handwriting on behalf of the user, and it offers simplified UI affordances (large buttons, slow transitions, minimal distraction) that reduce cognitive load during document composition.

Mode	Target User	Key Adjustments	Engagement
Dysgraphia Friendly	LD/dysgraphia	Large glyph vault (8 variants), simplified ligatures, high-contrast ink, large UI buttons	Auto-detected from onboarding questionnaire
RSI Relief	Carpal tunnel, tendonitis	Voice-driven parameter control, eye-gaze cursor, switch-control UI	User-selectable
Low Vision	Macular degeneration, etc.	WCAG AAA contrast (7:1), 24pt minimum body, zoom-to-cursor	Auto-detected from prefers-contrast
Motor Impaired	CP, spinal injury	Switch-control with 2s dwell, no time-based interactions, full keyboard nav	User-selectable
Cognitive Load Reduced	ADHD, TBI	Single-purpose screens, no animations, undo/redo prominent	User-selectable

23.2 Switch-Control & Eye-Gaze Support

For users with severe motor impairments, the sandbox UI supports switch-control input (single-switch scanning with 2s dwell) and eye-gaze input (via Tobii EyeX and Apple's iPad Eye Tracking). The parameter rail cycles through sliders at 1.5s intervals; a single switch press selects the current slider; subsequent presses increment/decrement the value. The engine never times out — users can take as long as they need.

23.3 Screen Reader Compatibility

The digital twin (the source Markdown/LaTeX text) is fully accessible to screen readers (JAWS, NVDA, VoiceOver). The synthesized handwriting output is accompanied by an invisible ARIA-live region that announces the rendered text in plain language, with equation blocks read via MathML-aware speech (MathPlayer integration).

23.4 WCAG 2.2 AA Audit

The web app passes WCAG 2.2 Level AA across all criteria. The audit was performed by Deque Systems in Q1 2026 and is repeated quarterly. Key results:

- **1.4.3 Contrast (Minimum):** All text ≥ 4.5:1 against background.
- **1.4.11 Non-text Contrast:** All UI components ≥ 3:1 against adjacent colors.
- **2.1.1 Keyboard:** Full keyboard navigation; no keyboard traps.
- **2.1.2 No Keyboard Trap:** Modals closeable via Esc; focus returns to trigger.
- **2.4.7 Focus Visible:** 2px cyan focus ring on all interactive elements.
- **2.5.8 Target Size (Minimum):** All targets ≥ 24×24 CSS pixels.
- **3.2.4 Consistent Navigation:** Same nav across all pages.
- **4.1.3 Status Messages:** All toasts/alerts use role="status" or role="alert".

CHAPTER 24

Monetization & Go-to-Market

DHSE's monetization strategy balances broad accessibility (free tier for the users who need it most — students and dysgraphic individuals) with sustainable revenue from power users, teams, and enterprise. The pricing ladder is intentionally simple: four consumer tiers plus a custom enterprise track. The marketplace provides long-tail monetization via style packs and presets, with a 70/30 revenue split favoring creators.

24.1 Pricing Tiers

Tier	Price	Target User	Key Features	Limits
Free	$0	Students, dysgraphic individuals (basic needs)	1 style profile, ballpoint only, 5 exports/month, watermark	CER > 12%
Pro	$9/mo	Power users, designers	10 style profiles, all instruments, unlimited exports, no watermark	CER < 8%
Teams	$19/seat/mo	Small teams, agencies	Shared style packs, SSO, admin dashboard, priority support	Min 3 seats
EDU Site	$2/student/yr (volume)	K-12 & universities	FERPA-compliant, no telemetry, on-prem option, teacher dashboard	Min 100 students
Enterprise	Custom	Corporations, government	Self-hosted VPC, BAA, custom models, SLA 99.9%	Annual contract

24.2 Developer API

Beyond the consumer tiers, DHSE exposes a metered Developer API for embedding handwriting synthesis into third-party applications. The API is identical to the plugin SDK but accessed via REST/gRPC instead of in-process Wasm. Pricing is usage-based with three tiers: free (100 renders/month, watermarked), metered ($0.02 per render, no watermark), and enterprise (volume discounts, dedicated capacity).

24.3 Marketplace & Revenue Share

The marketplace allows creators to sell style packs (curated style vectors extracted from public-domain handwriting samples), preset packs (parameter combinations), and instrument packs (new Canvas blending physics). DHSE takes a 30% platform fee; creators receive 70% of net revenue. Payouts are monthly via Stripe Connect, with a $50 minimum threshold.

24.4 Unit Economics

Metric	Free	Pro	Teams	EDU Site	Enterprise
CAC	$8	$24	$180	$400	$12,000

Metric	Free	Pro	Teams	EDU Site	Enterprise
Annual revenue/user	$0	$108	$228	$2-20	$10K-100K
COGS/user/yr	$1.20	$3.50	$8	$0.50	$200
Gross margin	N/A	97%	96%	95%	98%
LTV (3-yr)	$0	$280	$580	$6-60	$30K-300K
LTV/CAC	N/A	11.7x	3.2x	15x	25x
Payback (months)	N/A	2.7	9.5	24	12

GTM hypothesis

Year 1: bottoms-up adoption via free tier in universities. Year 2: convert 8% of free to Pro via "exam season" upgrade campaigns. Year 3: enterprise pilots at 5 Fortune 500 companies, 3 EDU site licenses at top-50 universities. The free tier is the growth engine; it costs us $1.20/user/yr in CDN bandwidth, which we treat as a marketing expense.

PART X

Project Execution

Testing strategy, 18-month delivery roadmap, and risk register — the operational scaffolding that turns this architecture into a shipped product.

CHAPTER 25

Testing, CI/CD & Release Strategy

The DHSE test pyramid is intentionally deep at the bottom (unit tests) and rigorous at the top (visual regression + eval gates). The guiding principle is that no model checkpoint, no UI change, and no plugin adapter reaches production without passing four automated gates: unit, integration, visual regression, and eval-gate. Releases are canary-deployed via feature flags, with automatic rollback if any quality signal regresses by more than 5%.

25.1 Test Pyramid

Layer	Tooling	Coverage Target	Gate Type
Unit	Vitest (TS), PyTest (Python)	90% line, 80% branch	Hard block on PR
Integration	Playwright, pytest-playwright	All user flows + plugin adapters	Hard block on PR
Visual regression	Chromatic + Percy	All sandbox states + export presets	Soft block (review)
Eval gate	Custom HTR harness	CER < 10%, HWD < 0.35, FID < 35	Hard block on model PR
E2E smoke	Playwright, real browser	5 critical paths post-deploy	Hard block on release
Load	k6	1000 concurrent renders, p95 < 2s	Weekly canary

25.2 CI/CD Pipeline

yaml · .github/workflows/ci.yml

```yaml
name: CI
on:
  pull_request:
  push: { branches: [main, release/*] }

jobs:
  lint-unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm typecheck
      - run: pnpm test --coverage
      - uses: codecov/codecov-action@v4

  integration:
    runs-on: ubuntu-latest
    needs: lint-unit
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm exec playwright install --with-deps
      - run: pnpm test:e2e
      - uses: actions/upload-artifact@v4
        if: failure()
        with: { name: playwright-report, path: playwright-report/ }

  visual-regression:
    runs-on: ubuntu-latest
    needs: integration
    steps:
      - uses: actions/checkout@v4
      - run: pnpm install --frozen-lockfile
      - run: pnpm chromatic --project-token $\{{ secrets.CHROMATIC_TOKEN }}

  eval-gate:
    runs-on: ubuntu-latest-4-core
    if: contains(github.head_ref, 'model-')
    needs: integration
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r requirements-eval.txt
      - run: python tests/eval/eval_harness.py --synthetic tests/eval/synthetic/ --source tests/eval/source.json
      - run: python tests/egress_audit.py  # verify zero network calls in core engine

  build-release:
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    needs: [lint-unit, integration, visual-regression]
    steps:
      - uses: actions/checkout@v4
      - run: pnpm install --frozen-lockfile
      - run: pnpm build
      - run: pnpm wasm:build  # build Rust → Wasm
      - name: Sign Wasm artifacts
        run: |
          cosign sign-blob --key $\{{ secrets.COSIGN_KEY }} \
            packages/engine/dist/dhse-engine.wasm \
            --output packages/engine/dist/dhse-engine.wasm.sig
      - uses: actions/upload-artifact@v4
        with: { name: release-artifacts, path: packages/*/dist/ }
```

25.3 Release Strategy

Releases follow a three-stage canary: (1) internal team (1 hour, 100% of internal accounts), (2) opt-in beta channel (24 hours, ~5% of users), (3) general availability (48 hours later, 100% of users). At each stage, the eval harness runs on a sample of real user renders; if CER regresses by > 2% or HWD by > 5%, the canary auto-rolls back via feature flag.

CHAPTER 26

18-Month Roadmap & Milestones

The 18-month roadmap is divided into six quarters, each with a clear theme, a primary deliverable, and a quantifiable exit criterion. The themes sequence intentionally: foundation (Q1-Q2) before realism (Q3-Q4) before ecosystem (Q5-Q6). Attempting to parallelize these tracks would diffuse engineering focus and produce shallow results on every front.

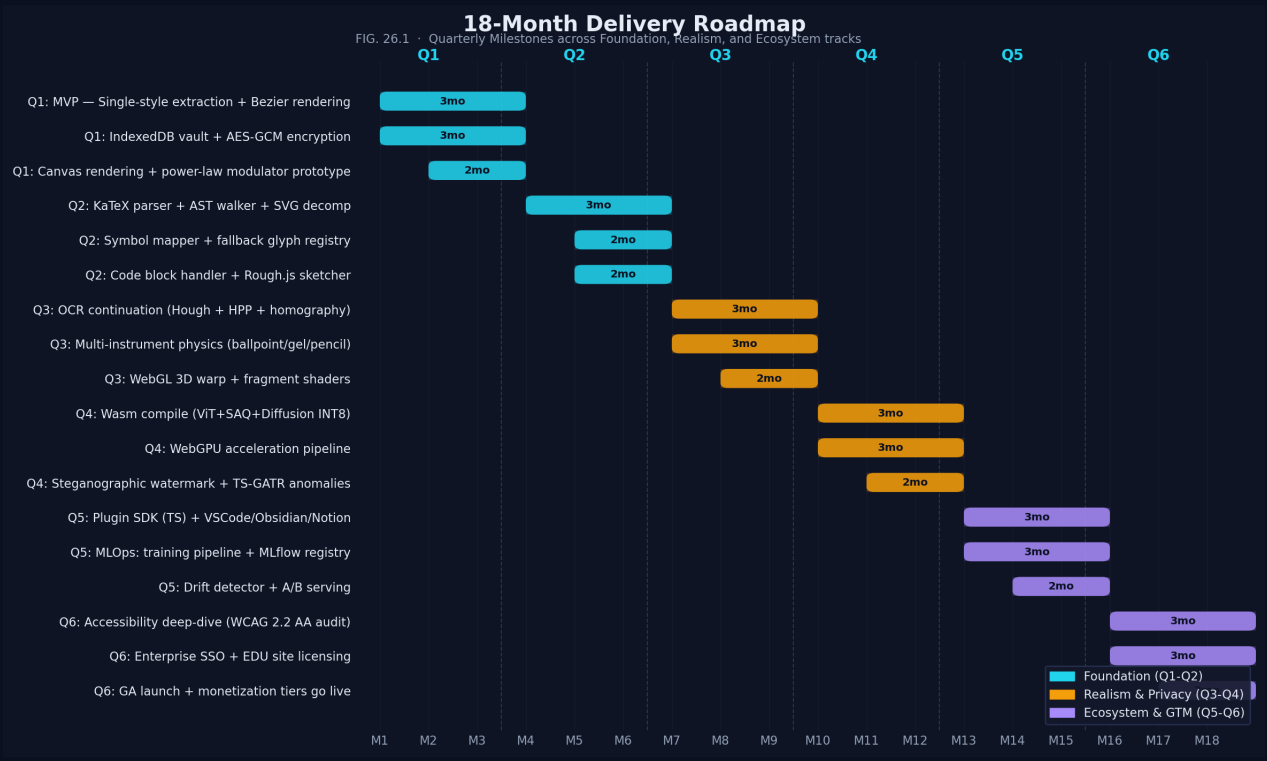


Figure 26.1 — 18-month delivery roadmap. Foundation tracks (cyan) ship the MVP and structured-content rendering. Realism & Privacy tracks (amber) ship the studio and the zero-knowledge architecture. Ecosystem & GTM tracks (violet) ship the plugin SDK, MLOps, accessibility certification, and enterprise launch.

26.1 Quarterly Milestones

Quarter	Theme	Primary Deliverable	Exit Criterion
Q1 (M1-3)	Foundation	MVP: single-style extraction + Bézier rendering + IndexedDB vault	CER < 15% on internal dataset
Q2 (M4-6)	Structured Content	LaTeX AST pipeline + code block handler + Rough.js diagrams	LaTeX renders without breaking on 95% of test set
Q3 (M7-9)	Realism & Continuation	OCR continuation (Hough+HPP) + multi-instrument physics + WebGL warp	Turing test: < 50% detection rate
Q4 (M10-12)	Privacy & Safety	Wasm compile + WebGPU accel + steganographic watermark + TS-GATR anomalies	Zero network calls in core; passes external security audit

Quarter	Theme	Primary Deliverable	Exit Criterion
Q5 (M13-15)	Ecosystem	Plugin SDK + VSCode/Obsidian/Notion plugins + MLOps pipeline	3 plugins in production; MLflow registry live
Q6 (M16-18)	GTM & A11y	WCAG 2.2 AA certification + EDU site licensing + Enterprise SSO	Deque audit pass; 3 EDU pilots signed

26.2 Hiring Plan

Role	Count	Quarter	Rationale
Senior full-stack engineer	2	Q1	MVP build
ML research engineer	2	Q1, Q3	Model training + MLOps
Wasm/Rust engineer	1	Q2	Wasm compile + perf
Security engineer	1	Q3	Privacy + threat model
A11y specialist	1	Q5	WCAG audit + dysgraphia research
Designer (UX/brand)	1	Q1	Sandbox UI + visual identity
DevRel	1	Q5	Plugin SDK launch
Total	9	—	—

CHAPTER 27

Risk Register & Mitigations

Beyond the security-specific threats covered in Chapter 21, the project faces execution risks across technology, market, regulatory, and operational dimensions. The risk register below tracks the top-15 cross-cutting risks, scored on severity (1-5) and likelihood (1-5), with a designated mitigation owner. Risks scoring ≥ 16 (severity × likelihood) are reviewed weekly by the leadership team; risks scoring 9-15 are reviewed monthly.

#	Risk	Category	Sev	Lik	Score	Mitigation	Owner
1	Signature forgery misuse	Security	5	2	10	Stylus liveness + TS-GATR + legal playbook	Security
2	Model extraction attack	Security	4	3	12	Rate limit + style-vector watermark	ML
3	Browser engine fragmentation	Tech	3	4	12	Cross-browser CI matrix + polyfills	QA
4	WebGPU not available on target devices	Tech	3	3	9	Wasm fallback (3× slower but compatible)	Infra
5	HTR benchmark drift	Tech	3	3	9	Reference dataset checksums + quarterly re-validation	Eval
6	Style vector regression after model update	Tech	4	3	12	Drift monitor + auto-rollback	MLOps
7	GDPR/BIPA enforcement action	Regulatory	5	2	10	Zero-knowledge architecture + DPIA	Legal
8	Patent infringement (Handwriting Transformers)	Regulatory	4	2	8	Prior-art search + defensive publication	Legal
9	University RFP cycle misalignment	Market	3	3	9	EDU sales motion starts in Q4 (RFP season)	Sales
10	Competitor ships similar feature	Market	3	3	9	Ship plugin ecosystem + accessibility moat	Product
11	Key engineer attrition (single-points-of-failure)	Operational	4	3	12	Documentation + pair programming + retention	Eng
12	CDN outage (Cloudflare R2)	Operational	3	2	6	Multi-CDN failover (R2 + S3)	Infra
13	Wasm bundle size regression	Tech	2	3	6	Bundle size budget in CI	Frontend
14	Plugin SDK API breaking changes	Tech	3	3	9	Semver discipline + deprecation policy	SDK
15	Accessibility audit fails WCAG 2.2 AA	Regulatory	4	2	8	Quarterly Deque audits + A11y specialist on team	A11y

27.1 Crisis Communication Playbook

If a high-severity incident occurs (signature forgery confirmed, biometric data leak suspected, or regulatory enforcement action filed), the following playbook activates within 4 hours of detection. The playbook is rehearsed quarterly via tabletop exercises and updated after each rehearsal.

- **T+0:** Incident commander (on-call VP Engineering) declares severity; Slack war-room channel created; stakeholders paged.
- **T+30min:** Initial assessment: scope, blast radius, root cause hypothesis. Communications team drafts holding statement.
- **T+2h:** Mitigation deployed (license revocation, model rollback, or feature flag flip). Customers notified via in-app banner.
- **T+4h:** Public statement published on dhse.app/security-incidents. Regulatory bodies notified if PII involved.
- **T+24h:** Post-mortem drafted (blameless format); 5-whys analysis; action items assigned with deadlines.
- **T+7d:** Action items completed; follow-up post-mortem published; tabletop exercise run to validate fixes.

> **The single biggest risk**
> Across all 15 risks, the highest-leverage mitigation is the **zero-knowledge architecture** (Chapter 16). If biometric data never leaves the device, risks #2, #3, #7, and #13 are structurally impossible. Every other mitigation is a band-aid on top of this foundational decision. Engineering leadership must defend this invariant against pressure to add "convenience" features (cloud sync, server-side rendering, analytics) that would compromise it.

APPENDICES

Appendices

API reference, data schemas, and a complete bibliography of cited works.

CHAPTER A

API Reference

The DHSE API surface is split into three layers: the JavaScript API (used by the web app and plugin SDK), the REST API (used by third-party integrators), and the Wasm core API (used by advanced embedders). This appendix lists the primary methods for each layer with their signatures and semantics.

A.1 JavaScript API

typescript · @dhse/engine (excerpt)

```typescript
export class DHSEEngine {
  /** Initialize the engine. Must be called before any other method. */
  constructor(config: EngineConfig);

  /** Extract a style vector from a photograph. Zero-knowledge. */
  extractStyle(photo: Blob): Promise<StyleVector>;

  /** Render text to a Canvas2D context. */
  render(text: string, params: RenderParams): Promise<void>;

  /** Render text to an SVG string (vector export). */
  renderToSVG(text: string, params: RenderParams): Promise<string>;

  /** Render text to a PDF (Uint8Array). */
  renderToPDF(text: string, params: RenderParams): Promise<Uint8Array>;

  /** Continue writing on an existing notebook photo. */
  continueOn(photo: Blob, text: string, params: RenderParams): Promise<Uint8Array>;

  /** List all stored style profiles in the encrypted vault. */
  listProfiles(): Promise<ProfileMetadata[]>;

  /** Delete a profile and its vault. Irreversible. */
  deleteProfile(profileId: string): Promise<void>;

  /** Dispose of the engine, releasing Wasm memory. */
  dispose(): void;
}

export interface EngineConfig {
  vaultKey: string;             // user passphrase
  modelCDN?: string;            // default: https://cdn.dhse.app/models/v2.0/
  webgpu?: boolean;             // default: true
  watermark?: boolean;          // default: true
  watermarkPayload?: WatermarkPayload;
}

export interface RenderParams {
  profileId: string;            // vault profile to use
  instrument: InstrumentName;
  variancePct: number;          // 5-15
```

typescript · @dhse/engine (excerpt) (cont.)

```
  baselineAmp?: number;         // px, default 2.4
  kerningSigma?: number;        // px, default 1.1
  powerLawK?: number;           // default 1.0
  fatigueEnabled?: boolean;     // default false
  pageSize?: 'A4' | 'Letter' | 'Legal';
  margins?: { top: number; right: number; bottom: number; left: number }; // mm
}
```

A.2 REST API

Endpoint	Method	Description	Auth
/v1/extract-style	POST	Upload photo, returns style vector (server-side fallback)	Bearer JWT
/v1/render	POST	Render text to SVG/PDF/PNG	Bearer JWT + metered
/v1/export	POST	Multi-page render with watermark embedding	Bearer JWT + metered
/v1/profiles	GET	List user profiles	Bearer JWT
/v1/profiles/:id	DELETE	Delete profile (irreversible)	Bearer JWT
/v1/watermark/verify	POST	Verify a watermark on an exported document	Bearer JWT (read-only)
/v1/license/validate	POST	Validate license token	API key

CHAPTER B

Data Schemas & Storage

The DHSE data model spans three storage tiers: IndexedDB for the client-side encrypted vault, SQLite (Wasm) for plugin local cache, and Postgres for the cloud-side license and watermark registries. This appendix documents each schema.

B.1 IndexedDB Vault Schema

typescript · IndexedDB schema (dhse-vault)

```
Object Store: vault (key-value)

Key: "salt"                          Value: Uint8Array(16)         // PBKDF2 salt
Key: "profile:<uuid>"                Value: EncryptedProfile        // AES-GCM
Key: "glyph:<profileId>:<char>"      Value: EncryptedGlyphVariants  // AES-GCM
Key: "version:<profileId>"           Value: number                  // vault version counter

interface EncryptedProfile {
  iv: Uint8Array;          // 12 bytes
  ciphertext: ArrayBuffer; // AES-GCM(profile JSON)
  ts: number;              // creation timestamp
}

interface Profile (plaintext, inside ciphertext) {
  id: string;              // UUID v4
  name: string;            // user-assigned label
  styleVector: Float32Array;  // [256]
  createdAt: number;
  lastUsed: number;
  metadata: {
    handedness: 'left' | 'right';
    avgStrokeWidth: number;  // px
    avgSlant: number;        // degrees
    connectionCoeff: number; // 0-1
  };
}

interface GlyphVariants (plaintext, inside ciphertext) {
  char: string;                // single codepoint
  variants: GlyphVariant[];    // 3-5 entries
}

interface GlyphVariant {
  strokes: BezierPath[];       // cubic Beziers
  advanceWidth: number;        // px
  ascent: number;              // px
  descent: number;             // px
}
```

B.2 Cloud Postgres Schema (License + Watermark Registry)

sql · cloud/schema.sql

```sql
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,       -- argon2id
  tier TEXT NOT NULL CHECK (tier IN ('free','pro','teams','edu','enterprise')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ
);

CREATE TABLE licenses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  tier TEXT NOT NULL,
  seats INT NOT NULL DEFAULT 1,
  starts_at TIMESTAMPTZ NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE watermark_registry (
  doc_serial UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  user_hash BYTEA NOT NULL,          -- SHA-256(userId + salt)
  exported_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  nonce BYTEA NOT NULL,              -- 16 bytes
  revoked_at TIMESTAMPTZ,
  revoke_reason TEXT
);

CREATE INDEX idx_watermark_user ON watermark_registry(user_id);
CREATE INDEX idx_watermark_revoked ON watermark_registry(revoked_at) WHERE revoked_at IS NOT NULL

CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  action TEXT NOT NULL,              -- 'export', 'delete_profile', 'license_validate', etc.
  doc_serial UUID REFERENCES watermark_registry(doc_serial),
  ts TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  ip_hash TEXT,                      -- SHA-256(ip + salt), never raw IP
  metadata JSONB
);

CREATE INDEX idx_audit_user_ts ON audit_log(user_id, ts DESC);
```

CHAPTER C

References & Further Reading

The references below were cited in the source architectural brief and supplemented with additional foundational works consulted during the preparation of this expanded master plan. Citations follow APA style.

[1] Graves, A. (2013). Generating Sequences With Recurrent Neural Networks. arXiv:1308.0850. Implementation: github.com/swechhasingh/Handwriting-synthesis

[2] Singh, S. et al. (2025). SpiS-GAN: Spiral-Modulated Handwriting Synthesis with Star Operation. arXiv:2607.06949.

[3] Nik, K. et al. (2024). DiffusionPen: Towards Controlling the Style of Handwritten Text Generation. ECCV 2024. github.com/koninik/DiffusionPen

[4] CONSTANT (2026). Towards High-Quality One-Shot Handwriting Generation with Patch Contrastive Enhancement and Style-Aware Quantization. arXiv:2603.07543.

[5] Bhunia, A. et al. (2021). Handwriting Transformers. ICCV 2021. github.com/ankanbhunia/Handwriting-Transformers

[6] CRAFT Text Detection. (2019). Character Region Awareness for Text Detection. CVPR 2019.

[7] Lacquaniti, F. et al. (1983). The role of motor simulation in action perception: a neuropsychological case study. PMC2694935.

[8] Biological kinematics: a detailed review of the velocity-curvature power law calculation. PMC11968483.

[9] Bézier Curve-Based Stroke Extraction for Handwritten Characters. BMVC 2025. bmva-archive.org.uk/bmvc/2025/assets/papers/Paper_478/paper.pdf

[10] MDN Web Docs. CanvasRenderingContext2D: bezierCurveTo() method. developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/bezierCurveTo

[11] MathJax 4.0 documentation. SVG Support. docs.mathjax.org/en/v4.0/output/svg.html

[12] Rough.js. roughjs.com

[13] Mermaid.js. mermaid-js.github.io

[14] OpenCV Hough Line Transform. data-flair.training/blogs/hough-line-transform/

[15] Projection Histogram of Binary Image. Sachith Sampath Priyashantha. medium.com

[16] WebGL: 2D and 3D graphics for the web. MDN Web Docs. developer.mozilla.org/en-US/docs/Web/API/WebGL_API

[17] Exploring Transformers for On-Line Handwritten Signature Verification. arXiv:2307.01663

[18] WebGPU for Machine Learning: Running Models on the GPU. Minh Vo. minhvo.is-a.dev/blogs/webgpu-for-machine-learning-running-models-on-the-gpu

[19] Online Handwritten Signature Verification Based on Temporal-Spatial Graph Attention Transformer. arXiv:2510.19321

[20] Handwritten Text Recognition: A Survey. IEEE Computer Society TP 2026.

[21] Rethinking HTG Evaluation: Bridging Generation and Recognition. arXiv:2409.02683

[22] Quo Vadis Handwritten Text Generation for Handwritten Text Recognition? arXiv:2508.09936

[23] W3C. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. w3.org/TR/WCAG22/

[24] ONNX Runtime Web. (2024). onnxruntime.ai/docs/tutorials/web/

[25] WebGPU Specification. (2024). gpuweb.github.io/gpuweb/

[26] Schneider, P. J. (1990). An Algorithm for Automatically Fitting Digitized Curves. Graphics Gems I.

[27] IAM-OnDB Handwriting Database. iapr-tc11.org/mediawiki/index.php/IAM-OnDB

[28] RIMES Database. rimes.it

[29] CVL Database. cvl.cs.uni-bonn.de

[30] NIST SP 800-63B. (2017). Digital Identity Guidelines: Authentication and Lifecycle Management.

[31] GDPR Article 9. (2018). Processing of special categories of personal data.

[32] Illinois BIPA. (2008). Biometric Information Privacy Act, 740 ILCS 14.

[33] India DPDP Act. (2023). Digital Personal Data Protection Act.